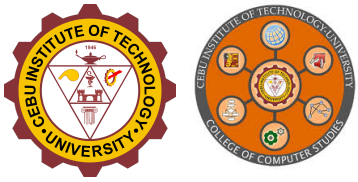
# CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES



**Software Design Description**

*for*

ELEMENTOPIA


**Submitted by:**

**2627-sem1-it411-53**

Ocampo, Danielle Maxine P.
Benolirao, Clyde C.
Ceniza, Jian Marc
Dabalos, Erica Y.
Dibdib, Wayne Kenji B.

**Submitted to:**

Engr. Ralph P. Laviste

Danielle Maxine P. Ocampo

Erica Y. Dabalos

Jian Marc Ceniza

Clyde C. Benolirao

Wayne Kenji B. Dibdib

| Date | Version | Description |
|---|---|---|
| May 15, 2026 | 1.0 | Initial creation of SDD document. |
| May 17, 2026 | 1.1 | Creating the introduction and overall description sections. |
| May 20, 2026 | 1.2 | Creating the diagrams. |
| Oct. 16, 2026 | 2.0 | Refactored the projects' purpose and mentioned in the scope the to be refactored features.<br>Added more acronyms and definitions, and references. |
| Oct. 17, 2026 | 2.0 | Post-MVP Architectural Refactoring: Incorporates validation findings so added FR-1.6, FR-2.4, and FR-3.4 from the SRS document, features to be developed based on validation findings. |
| | | |

[redacted]

This Software Design Document (SDD) outlines the architectural and component-level design of **Elementopia**, a gamified, web-based educational platform aimed at enhancing chemistry learning for high school students. It is submitted by Group 53 of IT332 under the College of Computer Studies at Cebu Institute of Technology – University as part of the academic requirements for the course.

The SDD details the system's overall structure, including user interface design, backend architecture, database schema, and integration strategies between components. The goal is to provide a clear blueprint for development, implementation, and future maintenance. All designs presented here are based on the specifications defined in the Software Requirements Specification (SRS) and have been adapted to reflect the team’s actual tools, workflows, and constraints.

This document is intended for instructors, project reviewers, and future developers who may build upon or maintain the system.

## Table of Contents


**1.** Introduction

1.1. Purpose 6

1.2. Scope 6

1.3. Definitions and Acronyms 7

1.4. References 7

2. Architectural Design

2.1. System Overview 8

2.2. Technology Stack 8

2.3. System Block Diagram 9

3. Detailed Design: Core Modules/Features

3.1. Module 1: Elemental Resonance Puzzle Module 10

3.2. Module 2: Assessment & Progress Module 33

3.3. Module 3: Opponent-Based Challenge Module 46

4. Cross-Cutting Design: Mobile Touch & Offline Resilience

4.1. Mobile Responsive Viewport and Touch Target Standard 57

4.2. Offline FIFO LocalStorage Sync Manager 57

# 1. Introduction

## *1.1. Purpose*

This Software Design Description (SDD) document details the architectural, component-level, and data design of Elementopia. This version of refactoring incorporates the technical directives identified during the multi-stakeholder MVP validation cycle. It translates the refactored functional and non-functional requirements from the SRS into concrete software designs, class interfaces, sequence flows, database schemas, and responsive UI layout specifications.

## *1.2. Scope*

Elementopia is a gamified, web-based educational system specifically bound to the DepEd Grade 8 and Grade 9 Science Curriculum (Matter and Chemical Bonding). The platform replaces passive memorization with active workbench element manipulation, contextual byproduct error diagnostics, automated failsafes, and real-time multiplayer challenges.

- **Core Features**:
    - **Elemental Resonance Domain Puzzle Rooms** — Students synthesize chemical compounds to trigger environmental reactions that clear physical obstacles blocking their progression.
    - **Meaningful Byproduct Diagnostics** — Replaces binary "Wrong" feedback with contextual chemistry-based guidance for incorrect compound combinations.
    - **Hazmat Protocol Failsafe** — Disables irrelevant elements after 5 failed attempts within 15 seconds to reduce random guessing.
    - **Randomized Compound Selection** — Compounds are drawn from a predefined pool per session to reduce repetitive memorization.
    - **Automated Session Performance Logging** — Tracks compound accuracy and completion speed to Supabase within under 2 seconds per session.
    - **Visual Mastery Dashboard** — Enables students to independently identify knowledge gaps without teacher intervention.
    - **Opponent-Based Challenge Module** — Peer-to-peer competitive matchmaking via auto-generated 5-digit access codes with connection time under 5 seconds.
    - **Speed-to-Compound Evaluation** — Real-time match winner determination based on reaction path efficiency via Supabase Realtime.
    - **Device-Agnostic Session Persistence** — Performance history is accessible across sessions as long as the same nickname is used.

The refactored design covers four primary architectural enhancements:

1. **Post-Synthesis Educational Debrief:** Integration of PostSynthesisDebriefModal.jsx to display chemical formulas, bond types, and real-world utility prior to barrier clearance.
2. **Centralized Diagnostic Pre-Test Cloud Sync:** Creation of an automated REST sync

endpoint (POST /api/telemetry/pretest) and PRETEST_DIAGNOSTIC schema in Supabase to eliminate client-side telemetry isolation.
3. **Multiplayer Disconnection & Forfeit Engine:** Implementation of HeartbeatTimeoutHandler.js within the Supabase Realtime channel to manage peer network dropouts gracefully without data loss.
4. **Mobile Viewport Touch-Target Optimization:** Implementation of a 48px x 48px minimum touch target standard and mobile-optimized CSS grid for tablet and smartphone viewports.

- **Technology Stack**:
    - **Frontend**: React.js (with Axios for API requests).
    - **Backend**: Spring Boot (Java).
    - **Database**: Supabase

## *1.3. Definitions and Acronyms*

- **SRS**: Software Requirements Specification
- **SDD**: Software Design Description
- **UI/UX**: User Interface/User Experience
- **PostgreSQL**: Relational Database Management System
- **API**: Application Programming Interface
- **JWT**: JSON Web Token
- **CRUD**: Create, Read, Update, Delete (for database operations)
- **Supabase:** An open-source Backend-as-a-Service (BaaS) platform that provides database, authentication, storage, and API services for applications.
- **DTO:** Data Transfer Object used for network payloads.
- **RDBMS:** Relational Database Management System (Supabase PostgreSQL).
- **FIFO:** First-In, First-Out queue used for offline telemetry caching.
- **REST API:** Application Programming Interface using REST architectural constraints.
- **WSS:** Two-way interactive communication channel over WebSocket.

## *1.4. References*

1. IEEE Computer Society. (2009). IEEE Standard for Information Technology — Systems Design — Software Design Descriptions (IEEE Std 1016-2009). Institute of Electrical and Electronics Engineers.
2. Department of Education (DepEd). (2016). K to 12 Science Curriculum Guide: Grade 9 Matter and Chemical Bonding. Pasig City, Philippines.
3. VMware Tanzu. (2026). Spring Boot Reference Documentation: Building RESTful APIs and Domain Controllers. https://spring.io/projects/spring-boot
4. Supabase Documentation. (2026). Realtime Channels, PostgreSQL Replication, and WebSockets. https://supabase.com/docs/guides/realtime
5. World Wide Web Consortium (W3C). (2018). Web Content Accessibility Guidelines (WCAG) 2.1: Success Criterion 2.5.5 Target Size (Minimum 48x48 CSS Pixels). https://www.w3.org/WAI/WCAG21/Understanding/target-size.html

# 2. Architectural Design

## *2.1. System Overview*

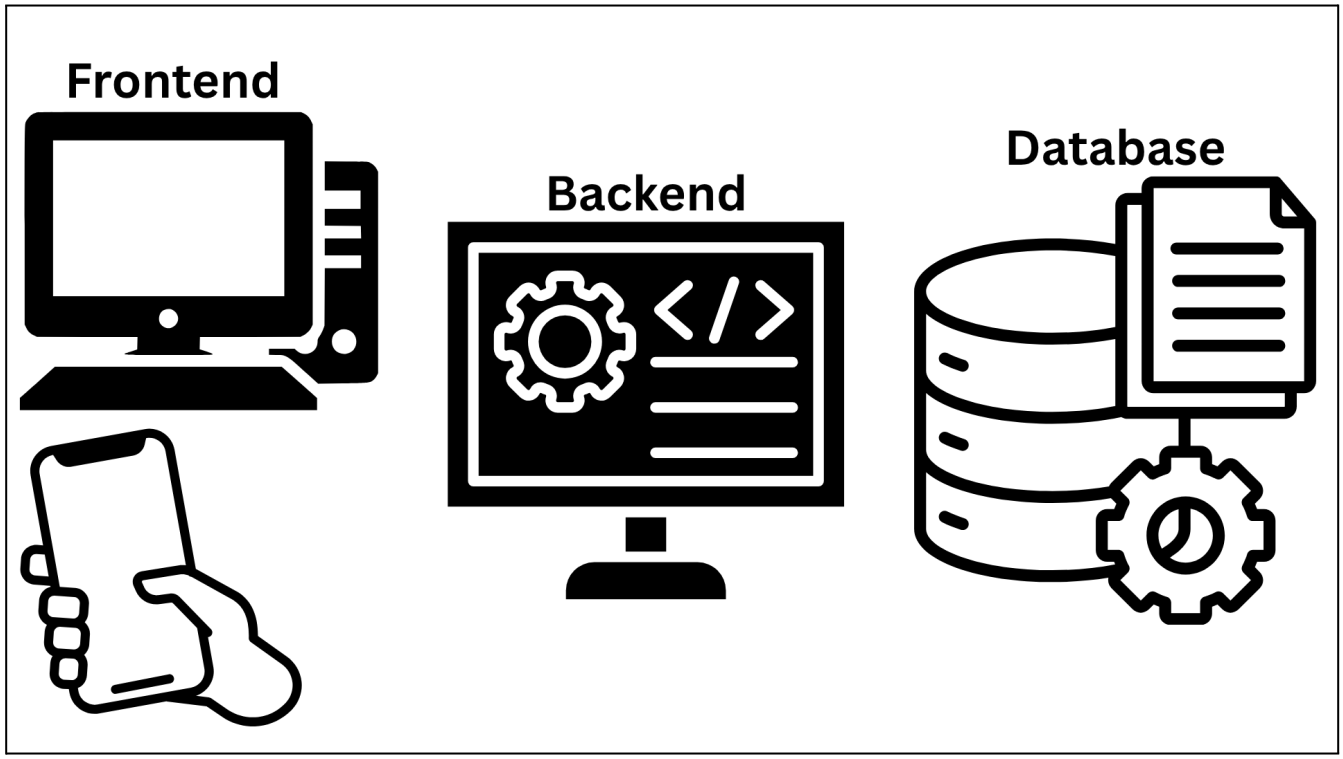
Elementopia follows a decoupled, three-tier client-server architecture designed for high visual responsiveness and persistent cloud analytics:

- **Presentation Tier (Frontend):** Built with React.js (Vite) utilizing modern functional component hooks, HTML5 Drag-and-Drop canvas APIs, and Tailwind CSS. The client manages local UI rendering, sound effects, and offline caching via browser localStorage.
- **Application Tier (Backend API):** Developed in Java using the Spring Boot framework. Exposes secure RESTful endpoints for formula validation, reaction byproduct lookups, brute-force failsafe monitoring, and session aggregation.
- **Data & Persistence Tier (Database):** Hosted on Supabase (PostgreSQL), providing relational tables for student session performance, historical progress, pre-test diagnostics, and competitive match lobbies. Real-time state synchronization is managed via Supabase Realtime WebSocket channels.

## *2.2. Technology Stack*

| Layer | Technology Component | Architectural Role |
|---|---|---|
| Frontend UI | React 18, Vite, Tailwind CSS | Reactive single-page application (SPA), responsive touch canvas workbench. |
| Frontend State | React Context API & LocalStorage | Local session management, offline FIFO queueing, and UI modal triggers. |
| Backend Service | Spring Boot 3.x (Java 17+) | REST API controllers, validation logic, sliding window evaluation. |
| Database | Supabase (PostgreSQL 15+) | Persistent relational storage for telemetry, room progression, and pre-tests. |
| Realtime Sync | Supabase Realtime (WebSockets) | Low-latency bidirectional state broadcasting for 1v1 multiplayer matches. |

## *2.3 System Block Diagram*




Description / Flow explanation:

The frontend communicates with the Spring Boot backend via asynchronous HTTPS REST requests for chemistry validation and telemetry logging. Spring Boot interacts with Supabase through Spring Data JPA / Hibernate for transactional persistence. For multiplayer duels, both student clients establish direct WebSocket connections with Supabase Realtime, bypassing server polling to achieve sub-second match state synchronization.

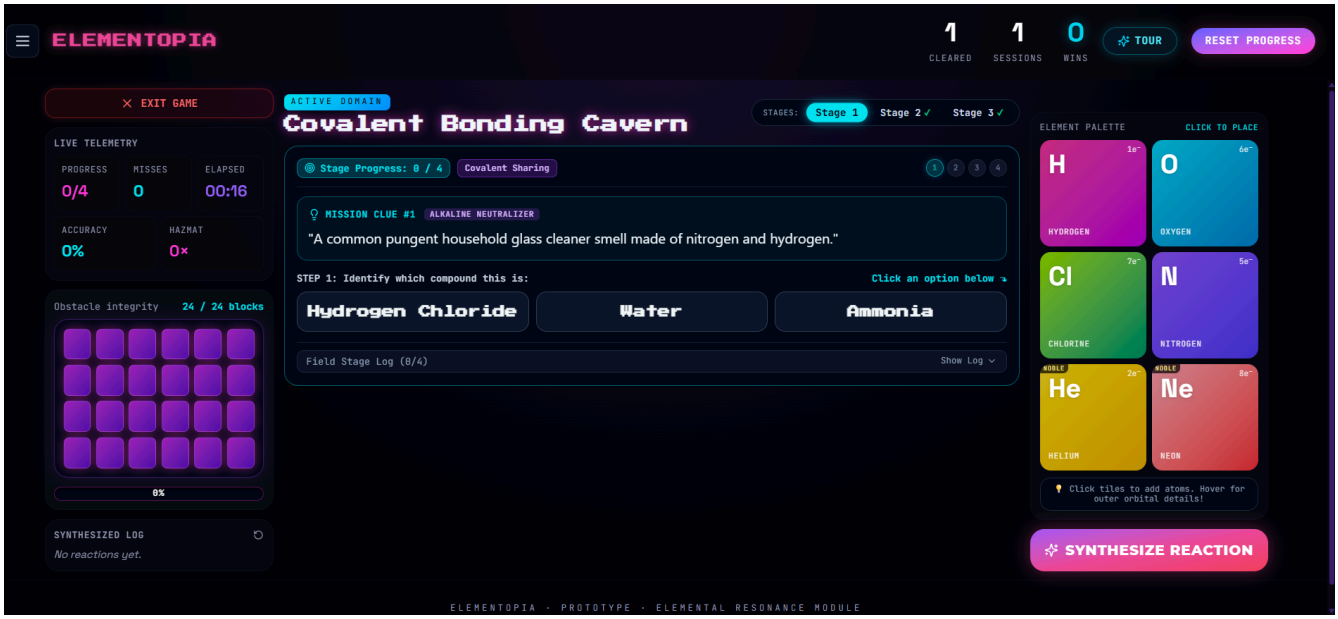
# 3. Detailed Design: Core Modules/Features

## *3.1. Module 1: Elemental Resonance Puzzle Module*

*The domain progression model is managed via domain.stages (Stages 1 through 3). Upon completing a stage, StageTransitionModal.jsx renders intermediate progress (Stage {currentStage} / 3 Debriefing) before loading the next stage's inventory and obstacle configuration.*

### *3.1.1 Elemental Resonance Domain Interaction*

- User Interface Design

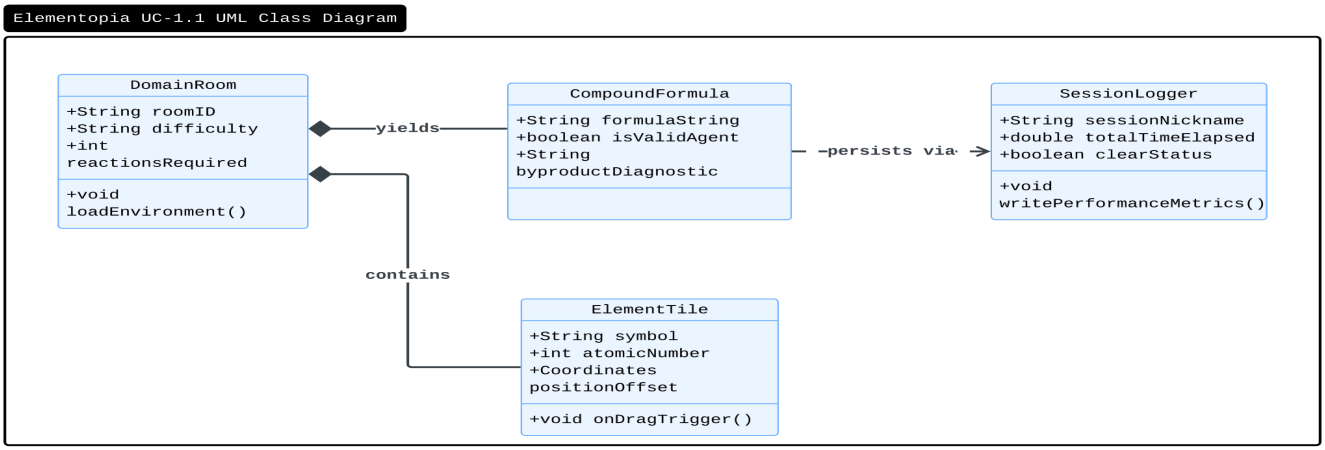



- Front-end component(s)
  - Component Name: ElementalDomainCanvas.jsx
    - Description and purpose: Inherits the core canvas rendering architecture of the original codebase (similar to ChemistrySimulation.jsx). It displays the graphical Domain environment, tracks drag-and-drop boundary interactions , and handles structural layout configurations.
    - Component type or format: React.js component with dynamic context hooks for touch-based tracking and visual asset state management.
  - Component Name: ActiveInventoryTray.jsx
    - Description and purpose: Renders the grid of chemical element tiles provided for the current challenge session. It hooks into state

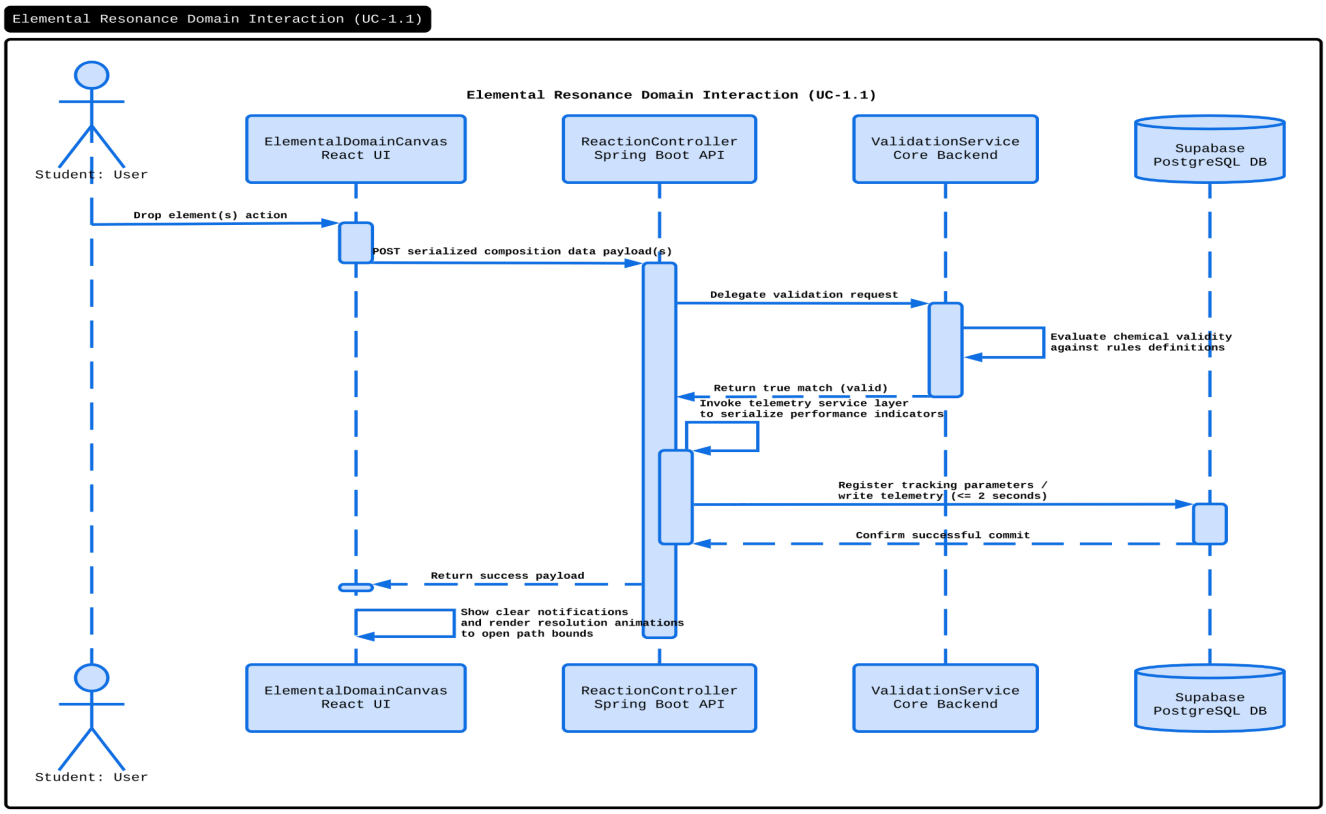
variables to visually disable or alter the interactable inventory selections when downstream processes require it.

- Component type or format: Functional React.js component fed directly by active gameplay state vectors.

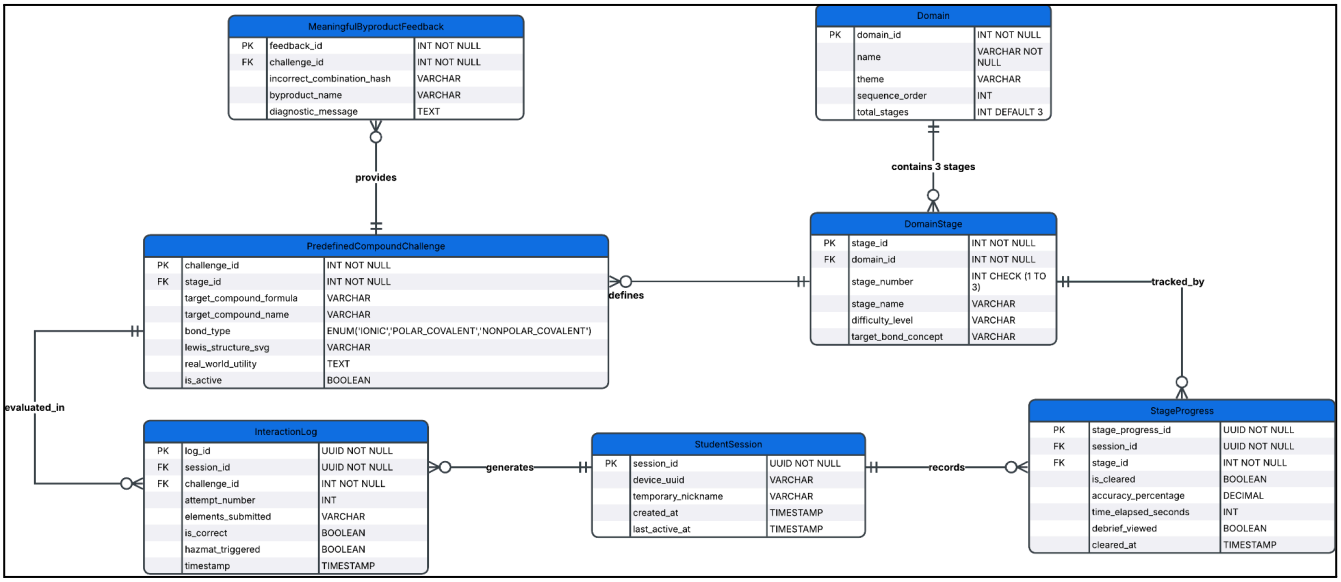
- Back-end component(s)
  - Component Name: ReactionController.java
    - Description and purpose: Exposes dedicated REST endpoints that accept element drop sequences from the client and maps inbound validation operations.
    - Component type or format: Spring Boot @RestController.
  - Component Name:ValidationService.java
    - Description and purpose: Evaluates the compound formula synthesis data sent by the client interface, validating the inputs against curriculum rules rather than traditional right/wrong text queries.
    - Component type or format: Spring Boot service component executing core business tier rules.
  - Component Name: TelemetryService.java
    - Description and purpose: Automatically captures completion speed and attempted variables from the session, routing structural transactions straight to cloud data collections.
    - Component type or format: Spring Boot service tracking database mutations via integrated JPA layers.
- Object-Oriented Components
  - Class Diagram



- Sequence Diagram:

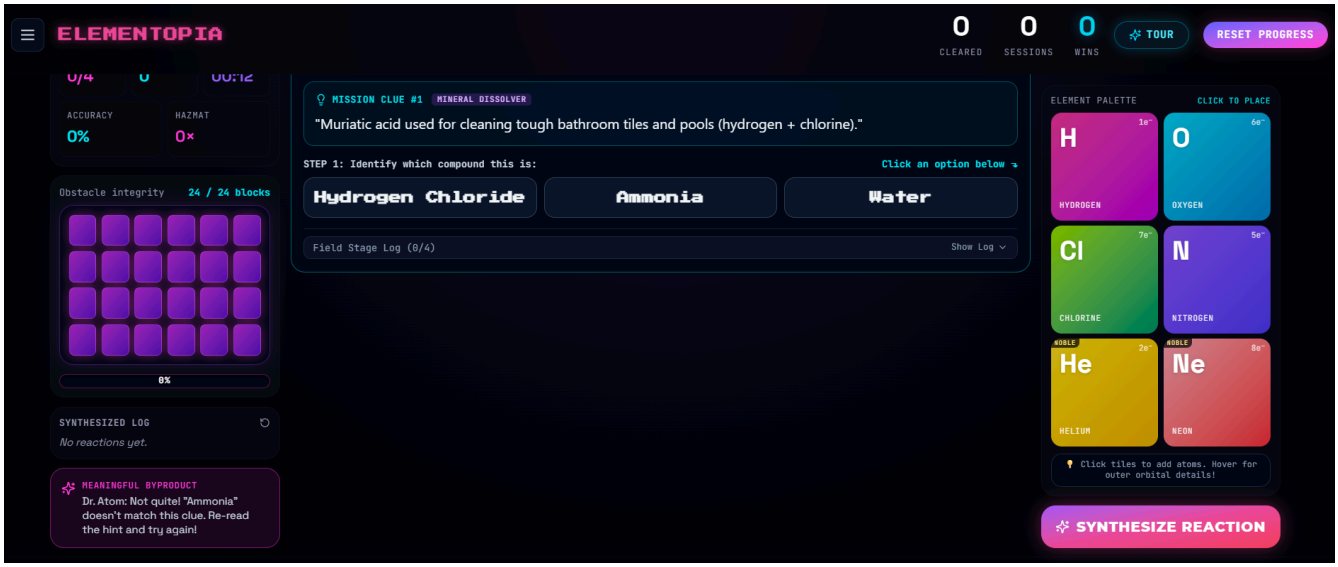



- Data Design
  - ERD or schema:



### *3.1.2 Meaningful Byproduct Diagnostic Feedback*

- User Interface Design







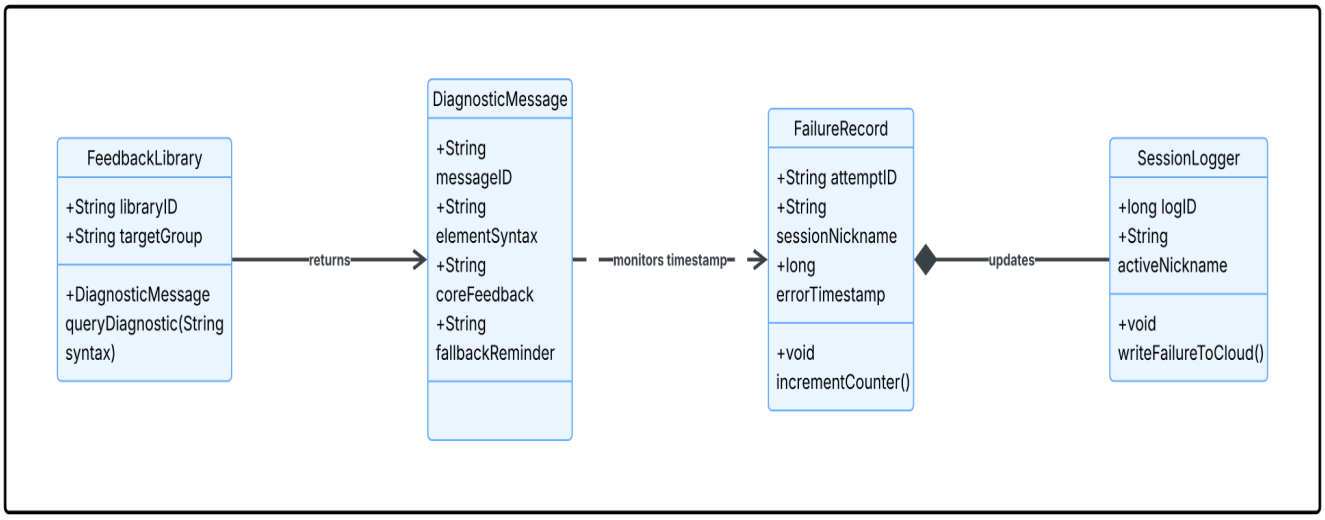





- Front-end component(s)
    - **Component Name:** `DiagnosticAlertOverlay.jsx`
        - **Description and purpose:** This component listens for invalid reaction responses from the backend validation layers. It conditionally overlays the active puzzle arena, rendering the chemistry-based feedback

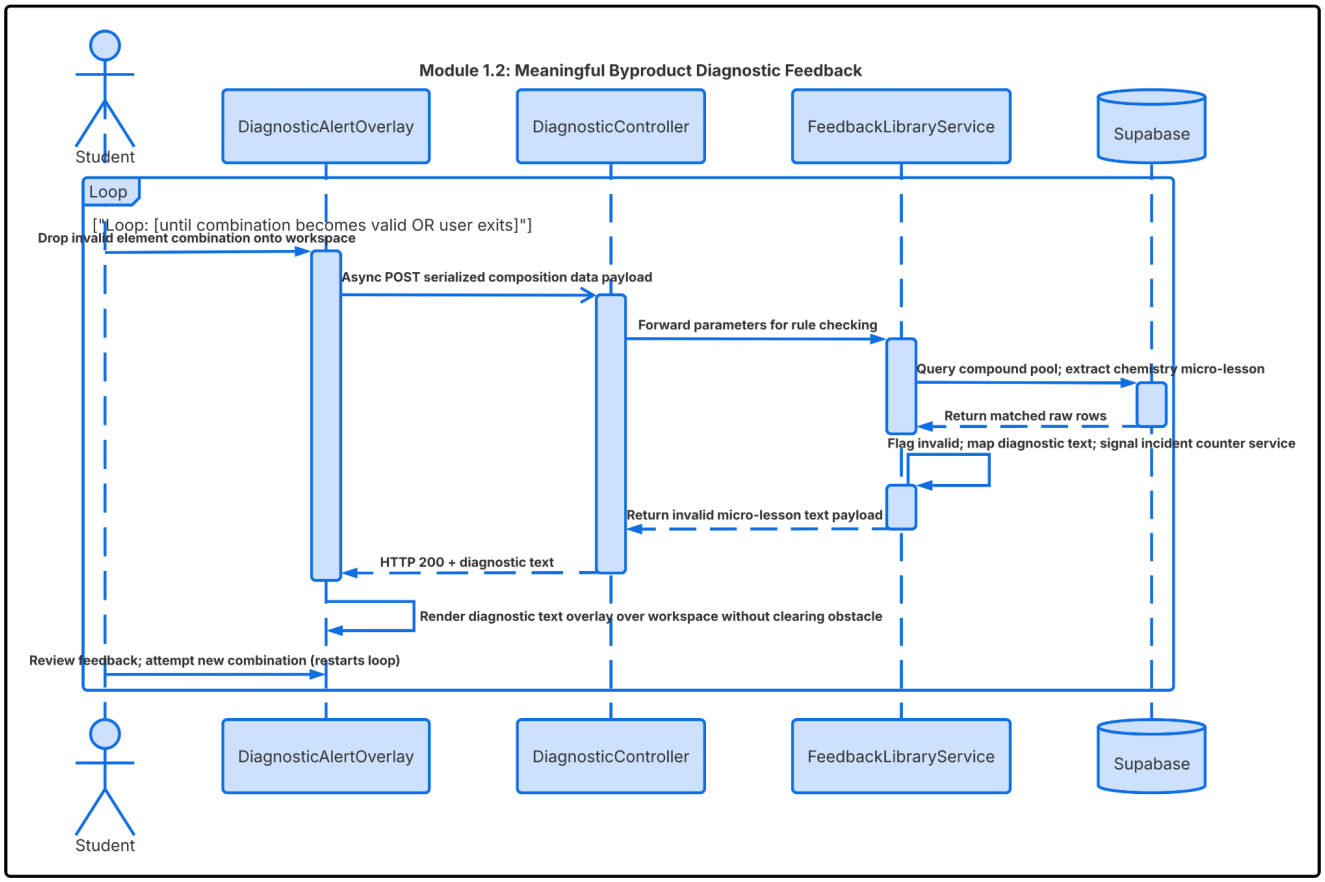
message (e.g., *"Inert Gas detected. No electron exchange occurred."*) directly inside the environment space, while advancing the session's hidden brute-force monitoring counter.
        - **Component type or format:** React.js functional component utilizing context state listeners and conditional canvas animation frames.
    - **Component Name:** `FeedbackLibraryContext.jsx`
        - **Description and purpose:** Provides a centralized frontend data state management context that maps general fallback chemistry principles (such as group or period attributes) if the backend returns a generic error code without a specific predefined reaction match.
        - **Component type or format:** React Context Provider (`.jsx`) wrapping the active learning Domain UI frame.
- Back-end component(s)
    - **Component Name:** DiagnosticController.java
        - **Description and purpose:** Exposes the backend API routing paths tasked with receiving combination inputs and handling error payload deliveries when validation tasks return failures.
        - **Component type or format:** Spring Boot @RestController managing REST endpoints over HTTPS channels.
    - **Component Name:** FeedbackLibraryService.java
        - **Description and purpose:** Evaluates the incorrect combination syntax and queries the system's predefined chemical properties library. If a precise relationship match exists, it retrieves the corresponding micro-lesson; otherwise, it dynamically generates a general periodic group or period principle reminder.
        - **Component type or format:** Spring Boot @Service component executing business rule diagnostic logic.
    - **Component Name:** IncidentCounterService.java
        - **Description and purpose:** Automatically logs every individual failed attempt and its corresponding millisecond timestamp directly to the session history cache. It increments the operational telemetry counters that feed into the background sliding-window brute-force failsafe monitor.
        - **Component type or format:** Spring Boot @Service handling transactional application state updates.
- Object-Oriented Components
    - Class Diagram:

Elementopia UC-1.2 Class Diagram (Three-Tier Cloud)

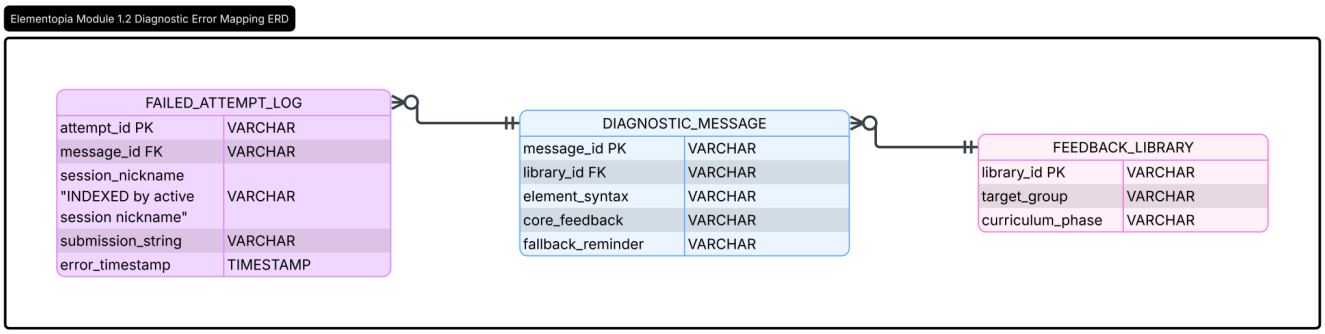



- Sequence Diagram:

Module 1.2: Diagnostic Feedback Loop

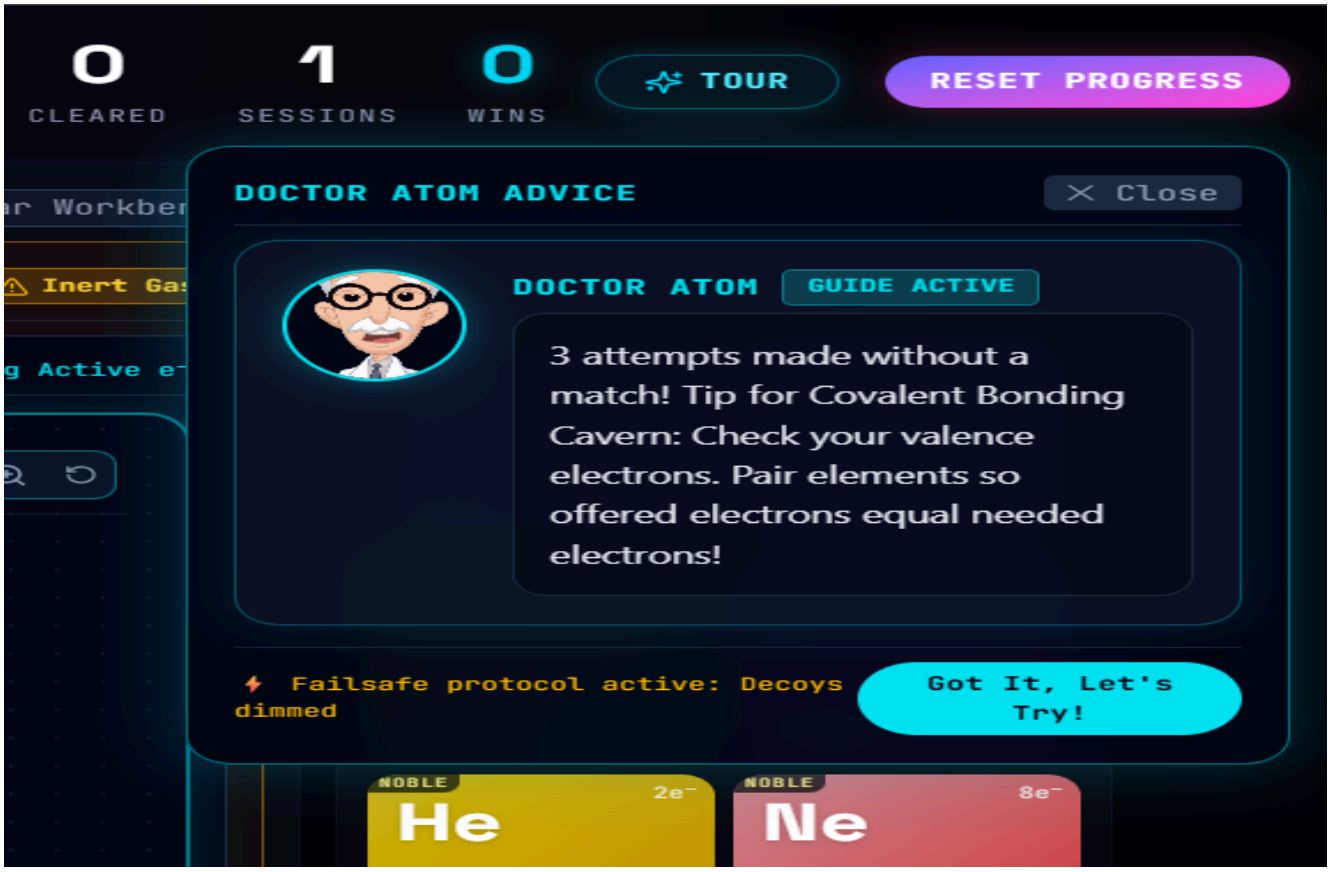


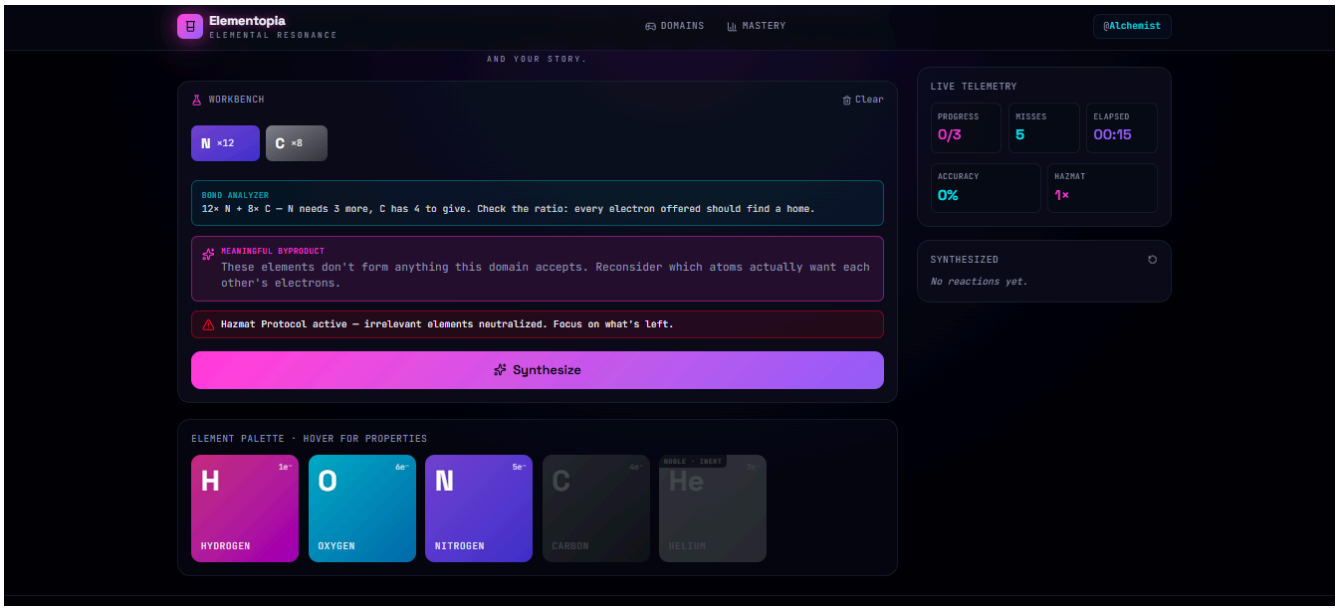
- Data Design
    - ERD or schema:




### *3.1.3 Brute-Force Failsafe*

- User Interface Design

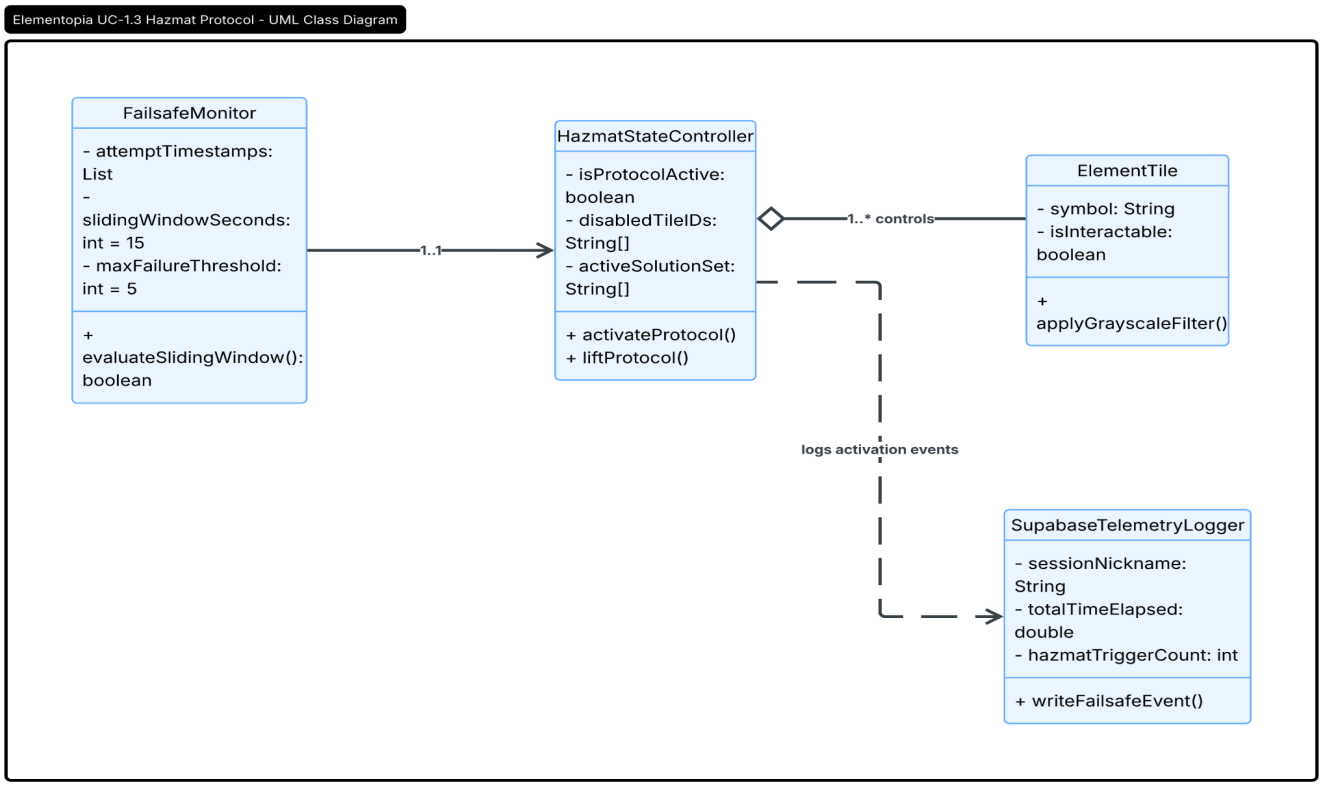





- Front-end component(s)
    - **Component Name:** HazmatOverlayController.jsx
        - **Description and purpose:** This component monitors the local interaction canvas state and intercepts touch/click events when the brute-force conditions are violated. When triggered, it visually grays out and locks all element tiles except those tagged as part of the correct solution path, restricting the student's interaction field to prevent aimless guessing.
        - **Component type or format:** High-order React.js component wrapper applying conditional CSS grayscale filters and state-driven pointer-events locking.
    - **Component Name:** SystemVolatilityBanner.jsx
        - **Description and purpose:** Renders an in-universe alert message text overlay (e.g., *'System Volatility High. Hazardous elements isolated.'*) across the screen immediately upon protocol activation, disappearing only after a valid compound synthesis lifts the intervention.
        - **Component type or format:** Child React.js presentational component triggered by background global state context hooks.
- Back-end component(s)
    - **Component Name:** FailsafeMonitorController.java
        - **Description and purpose:** Provides the API endpoint paths to stream security-focused state changes and notify the system layers of brute-force intervention statuses.
        - **Component type or format:** Spring Boot @RestController handling custom state update payloads.

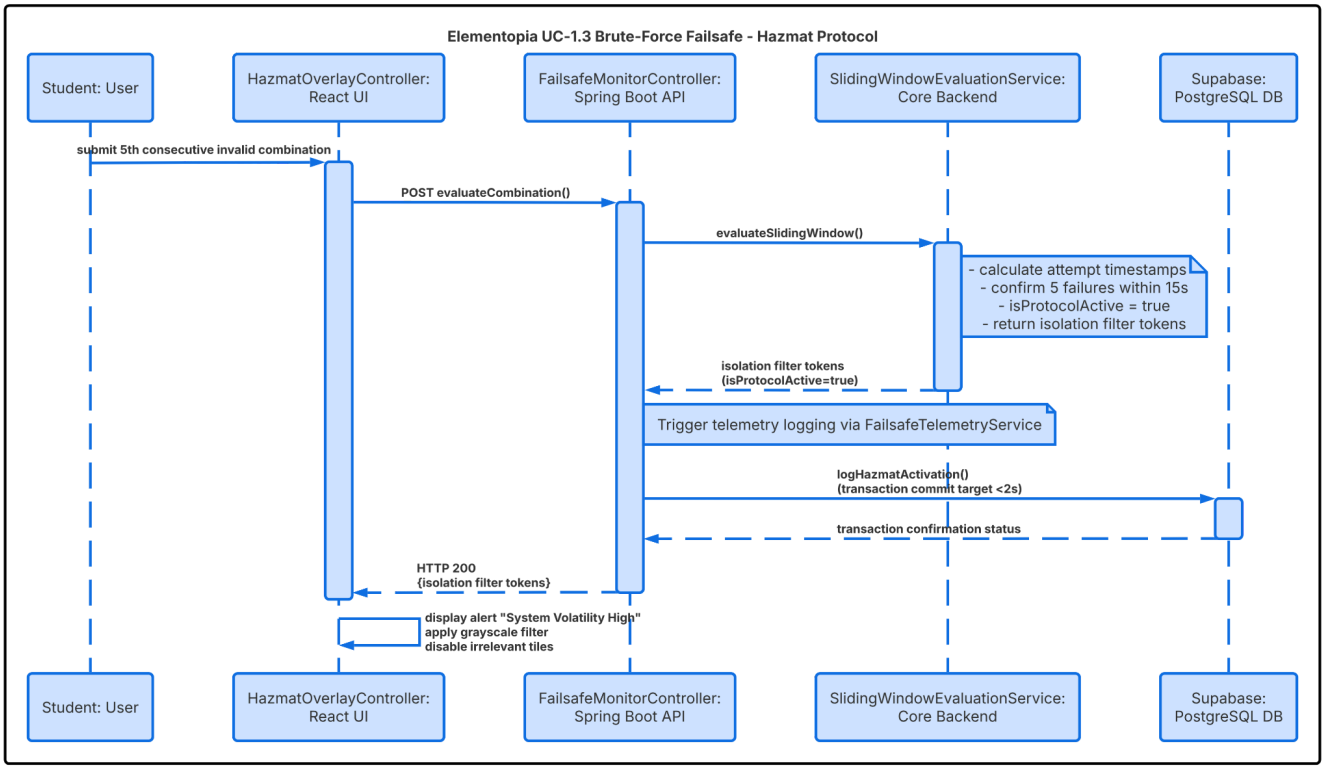
- **Component Name:** SlidingWindowEvaluationService.java
  - **Description and purpose:** Runs background verification loops evaluating transaction logs. It analyzes the timestamps of incoming validation requests from the same user nickname, calculating whether 5 or more failed combinations occurred within a 15-second sliding window. If true, it alters the active session context flag to enable the protocol.
  - **Component type or format:** Spring Boot @Service component executing business rules logic equations.
- **Component Name:** FailsafeTelemetryService.java
  - **Description and purpose:** Registers the precise activation and deactivation metrics of the Hazmat Protocol, logging the timestamp parameters safely to cloud storage under 2 seconds.
  - **Component type or format:** Spring Boot @Service controlling transaction mapping data bindings.

- Object-Oriented Components
  - Class Diagram:



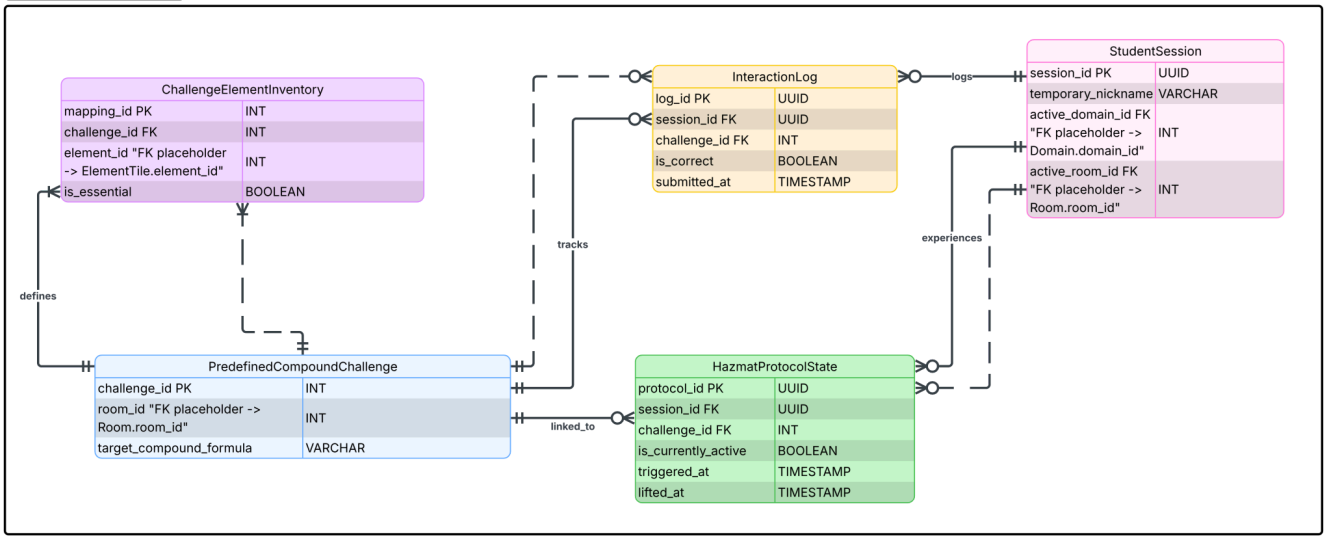
- Sequence Diagram:

Elementopia UC-1.3 Brute-Force Failsafe - Hazmat Protocol



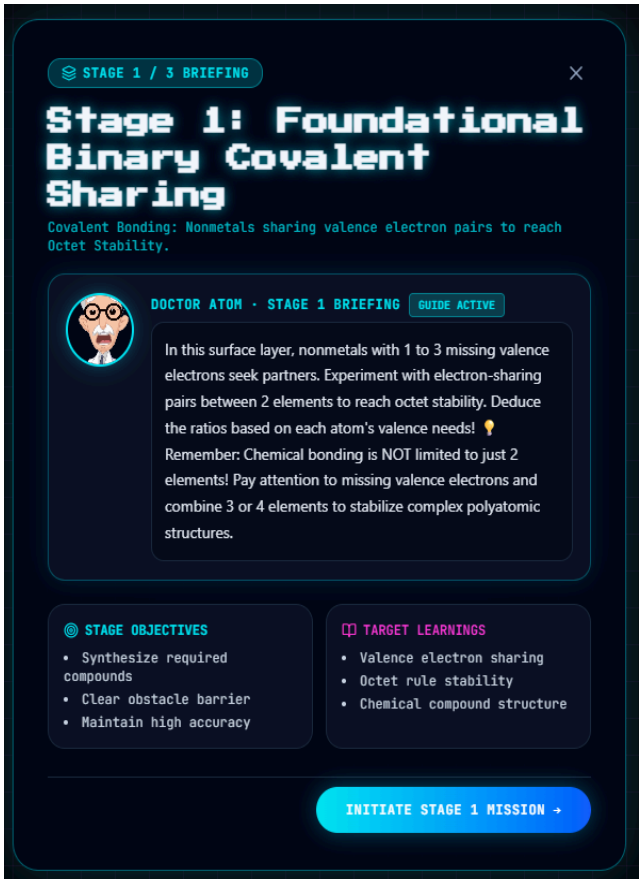

- Data Design
  - ERD or schema:

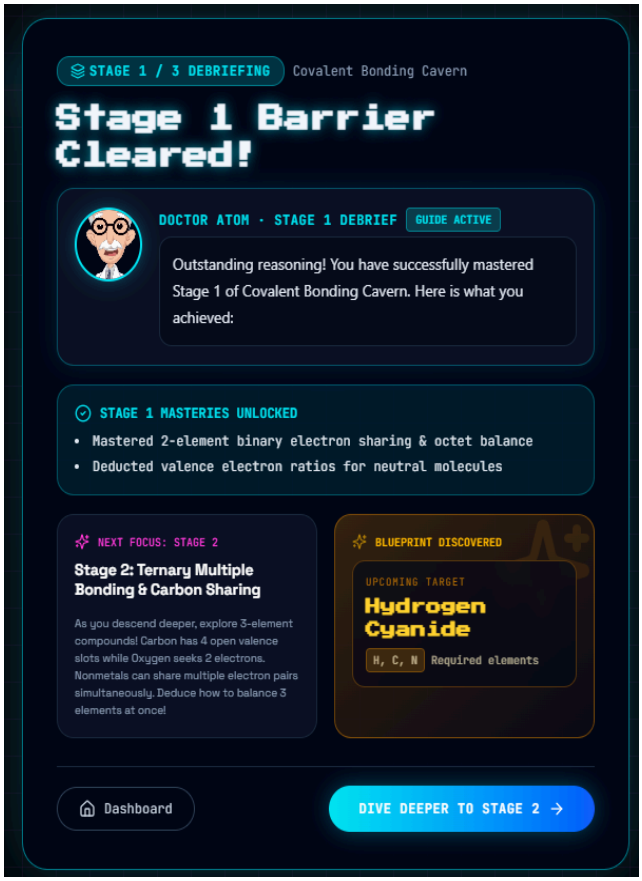
Elementopia 1.3: Hazmat Protocol ERD



### *3.1.4 Learning Room Progression*

- User Interface Design

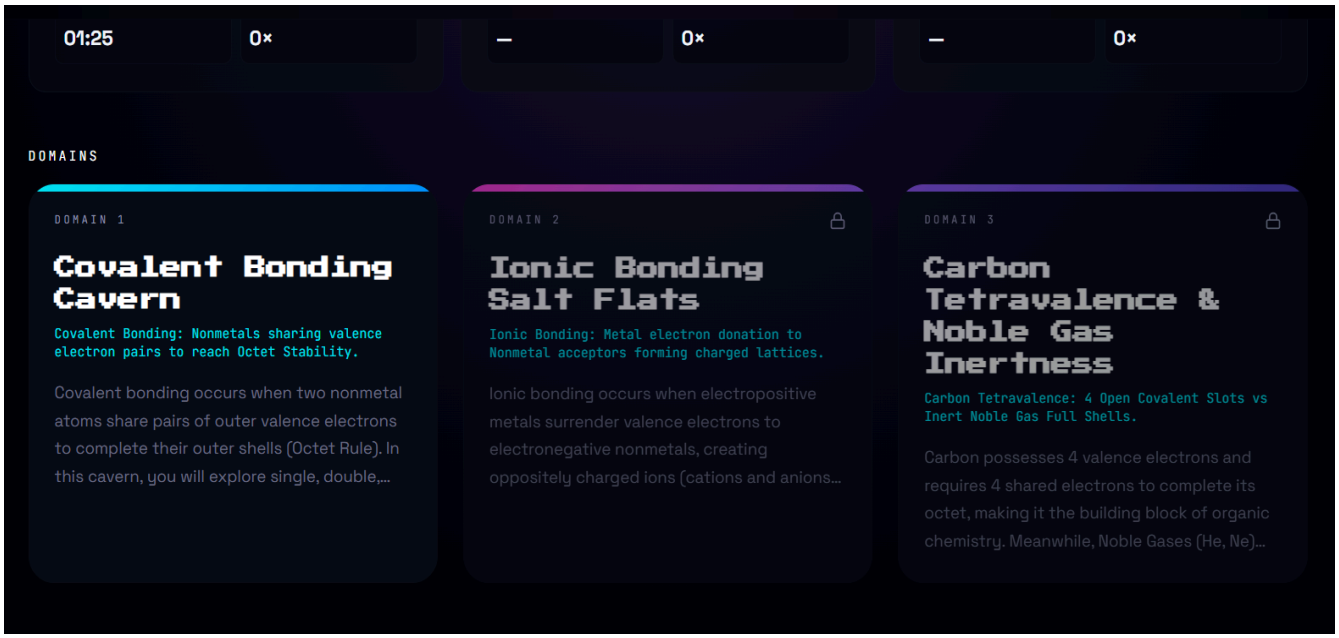



STAGE 2 / 3 DEBRIEFING
Covalent Bonding Cavern
Stage 2 Barrier Cleared!
DOCTOR ATOM · STAGE 2 DEBRIEF
GUIDE ACTIVE
Outstanding reasoning! You have successfully mastered Stage 2 of Covalent Bonding Cavern. Here is what you achieved:
STAGE 2 MASTERIES UNLOCKED
• Mastered 3-element ternary bonding & multiple shared electron pairs
• Deducted balance rules for polyatomic ion structures
NEXT FOCUS: STAGE 3
Stage 3: Polyatomic Organic Core (Organic Backbones)
Entering the domain core, Carbon chains link together into essential organic backbones. Combine Carbon, Hydrogen, Nitrogen, and Oxygen in precise ratios so every valence electron finds a partner!
BLUEPRINT DISCOVERED
UPCOMING TARGET
Urea
C, H, N, O
Required elements
Dashboard
DIVE DEEPER TO STAGE 3 →

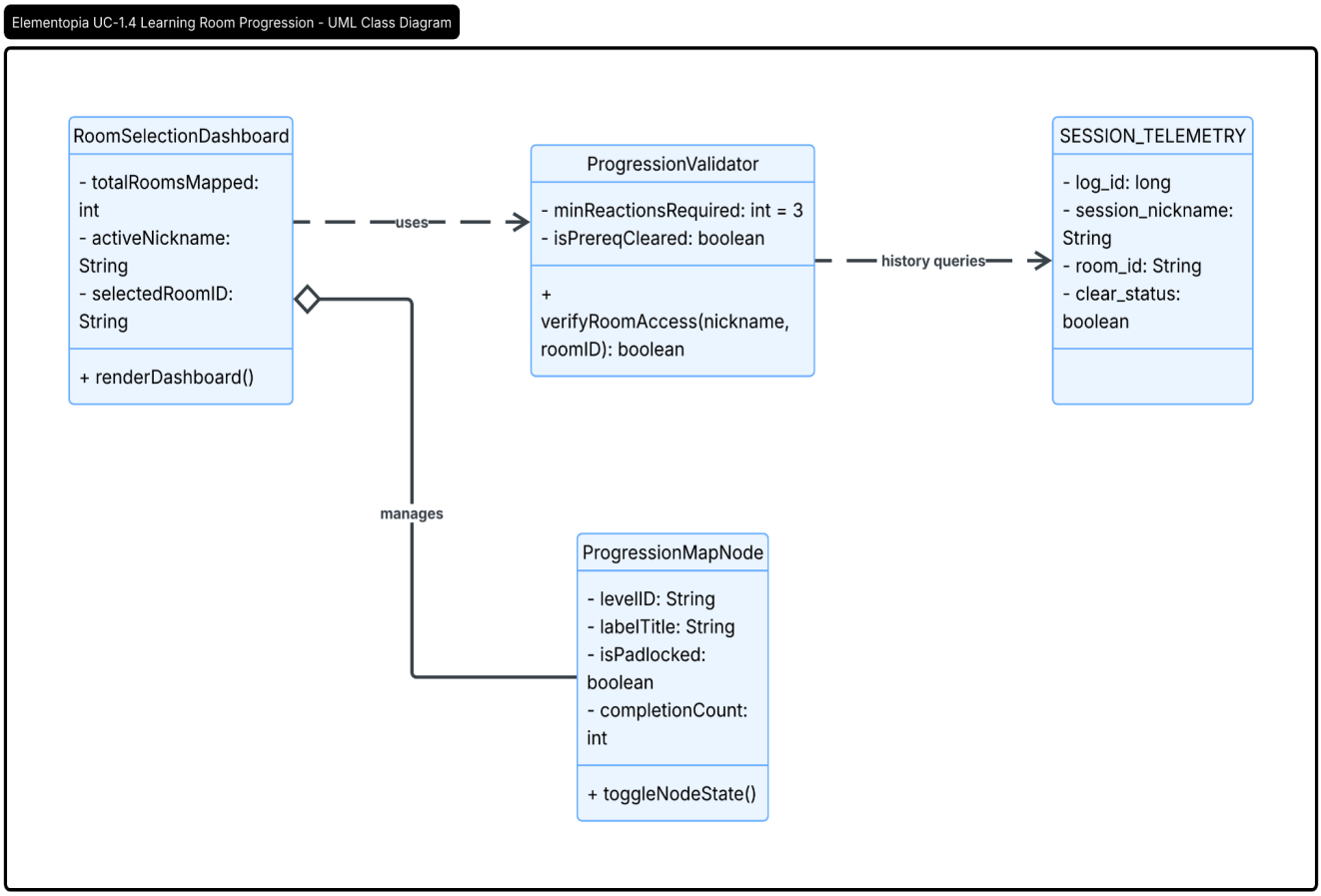

STAGE 2 / 3 DEBRIEFING
Covalent Bonding Cavern
Stage 2 Barrier Cleared!
DOCTOR ATOM · STAGE 2 DEBRIEF
GUIDE ACTIVE
Outstanding reasoning! You have successfully mastered Stage 2 of Covalent Bonding Cavern. Here is what you achieved:
STAGE 2 MASTERIES UNLOCKED
• Mastered 3-element ternary bonding & multiple shared electron pairs
• Deducted balance rules for polyatomic ion structures
NEXT FOCUS: STAGE 3
Stage 3: Polyatomic Organic Core (Organic Backbones)
Entering the domain core, Carbon chains link together into essential organic backbones. Combine Carbon, Hydrogen, Nitrogen, and Oxygen in precise ratios so every valence electron finds a partner!
BLUEPRINT DISCOVERED
UPCOMING TARGET
Urea
C, H, N, O
Required elements
Dashboard
DIVE DEEPER TO STAGE 3 →


STAGE 3 / 3 BRIEFING
Stage 3: Polyatomic Organic Core (Organic Backbones)
Covalent Bonding: Nonmetals sharing valence electron pairs to reach Octet Stability.
DOCTOR ATOM · STAGE 3 BRIEFING
GUIDE ACTIVE
Entering the domain core, Carbon chains link together into essential organic backbones. Combine Carbon, Hydrogen, Nitrogen, and Oxygen in precise ratios so every valence electron finds a partner! 💡 Remember: Chemical bonding is NOT limited to just 2 elements! Pay attention to missing valence electrons and combine 3 or 4 elements to stabilize complex polyatomic structures.
STAGE OBJECTIVES
• Synthesize required compounds
• Clear obstacle barrier
• Maintain high accuracy
TARGET LEARNINGS
• Valence electron sharing
• Octet rule stability
• Chemical compound structure
INITIATE STAGE 3 MISSION →



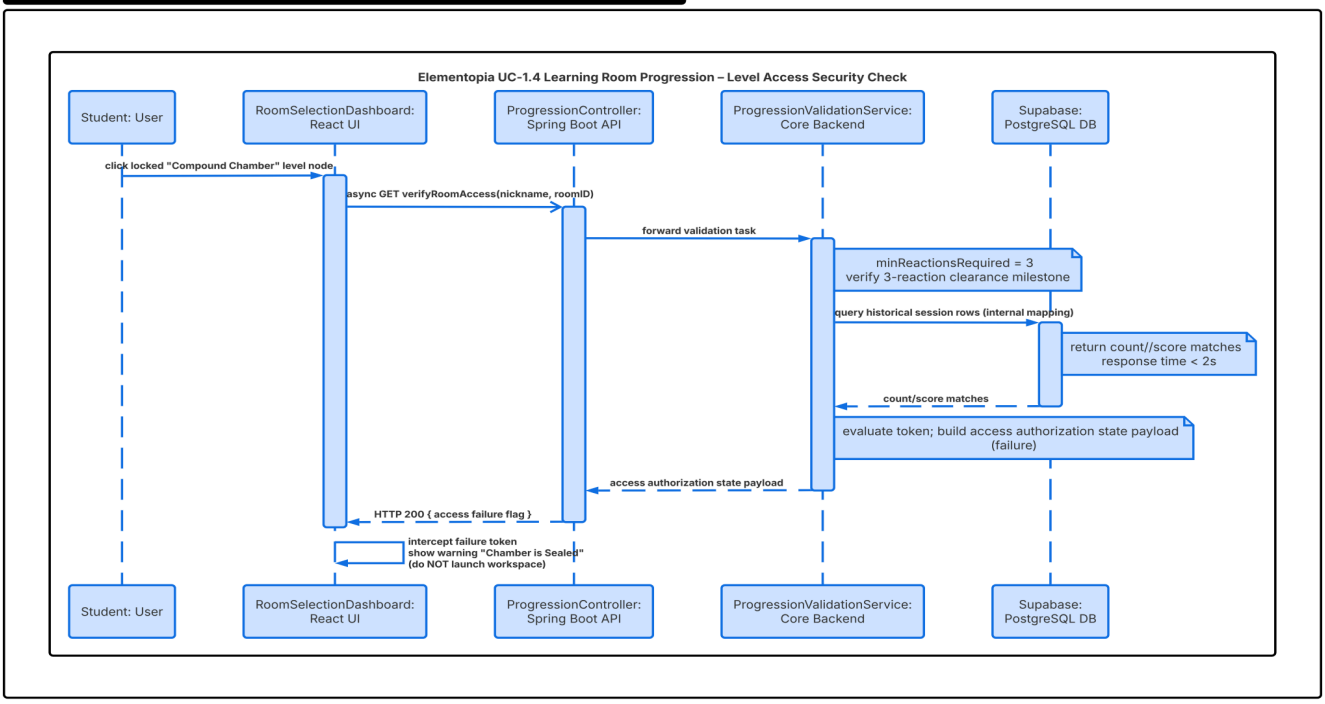

- Front-end component(s)
    - **Component Name:** RoomSelectionDashboard.jsx
        - **Description and purpose:** Renders the map screen layout showing all foundational Element Rooms and advanced Compound Chambers. It queries the user profile state, reads locked or unlocked conditions, and intercepts unauthorized pointer selection attempts on sealed rooms.
        - **Component type or format:** React.js functional view handling screen-width scaling from 360px to 1920px.
    - **Component Name:** ProgressionMapNode.jsx
        - **Description and purpose:** Renders individual level selector nodes displaying completion statuses, structural metadata labels, and visual padlock state badges.
        - **Component type or format:** Child React.js presentational component.
- Back-end component(s)
    - **Component Name:** ProgressionController.java
        - **Description and purpose:** Exposes endpoints to check lock configurations, parse student entry commands, and return initial puzzle session setups.
        - **Component type or format:** Spring Boot @RestController.
    - **Component Name:** ProgressionValidationService.java
        - **Description and purpose:** Verifies if a student meets prerequisite sequence rules (e.g., minimum 3 completed reactions in foundational rooms) before unlocking subsequent levels.

        - **Component type or format:** Spring Boot @Service component executing business rules logic.
    - **Component Name:** ProgressionTelemetryService.java
        - **Description and purpose:** Streams unlock records and updated cleared milestones directly to database instances under 2 seconds.
        - **Component type or format:** Spring Boot @Service controlling transaction mapping data bindings.
- Object-Oriented Components
    - Class Diagram:



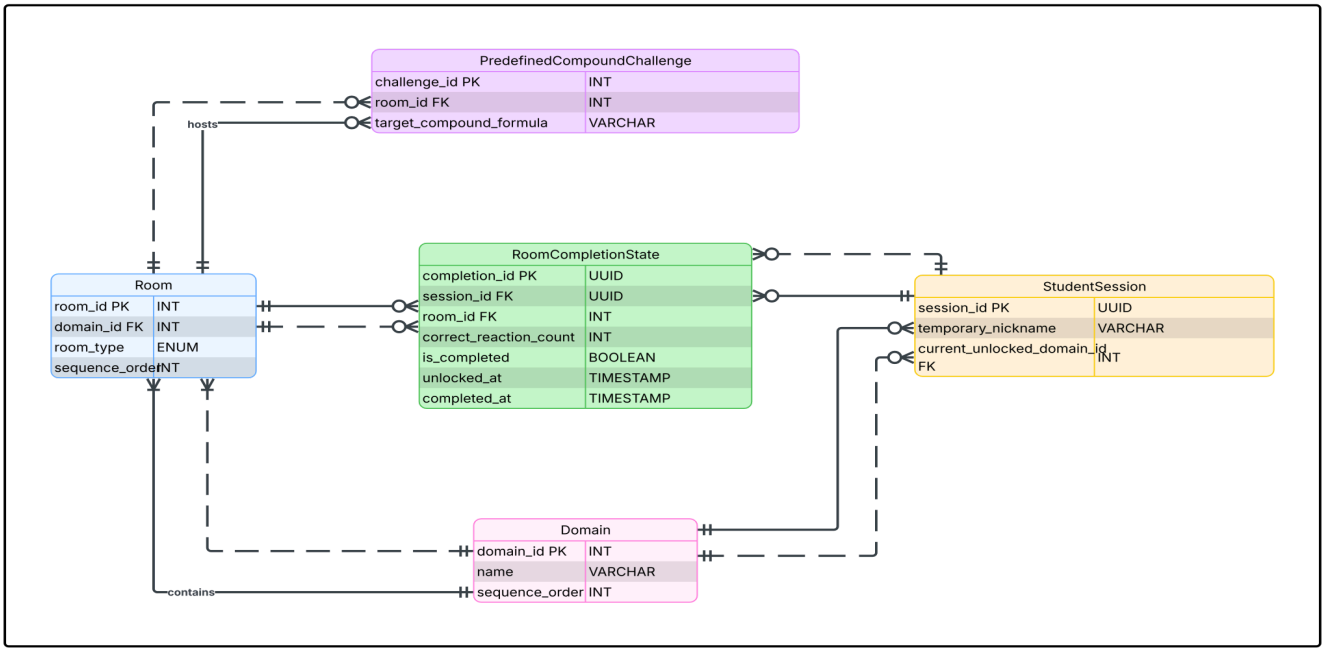
- Sequence Diagram:

Elementopia UC-1.4 Learning Room Progression – Level Access Security Check



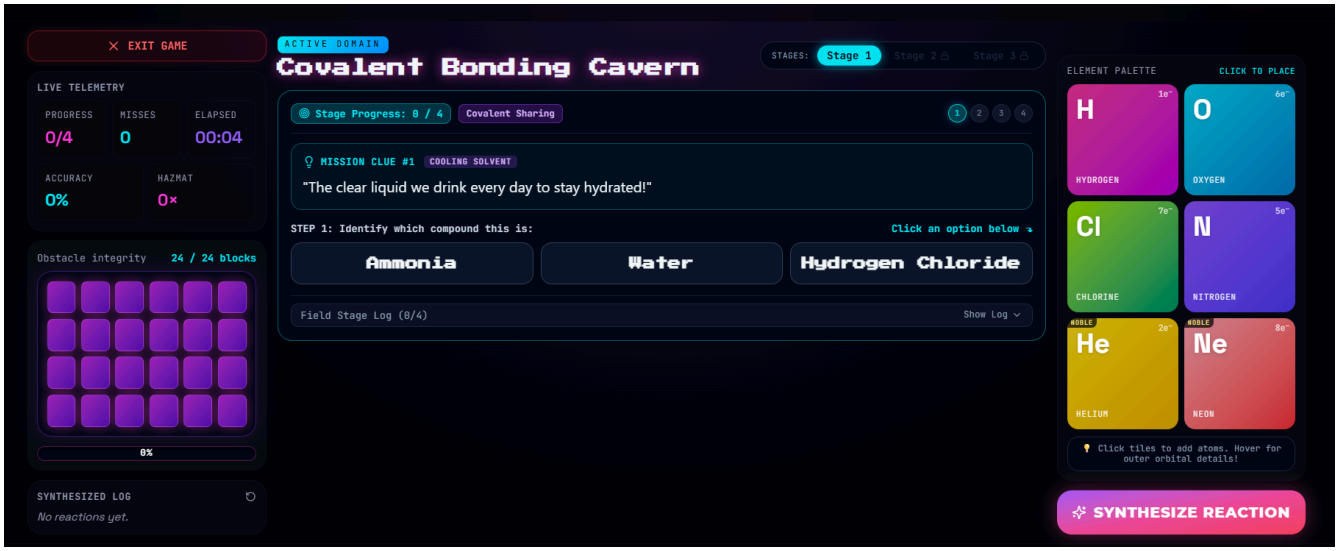

- Data Design
  - ERD or schema:

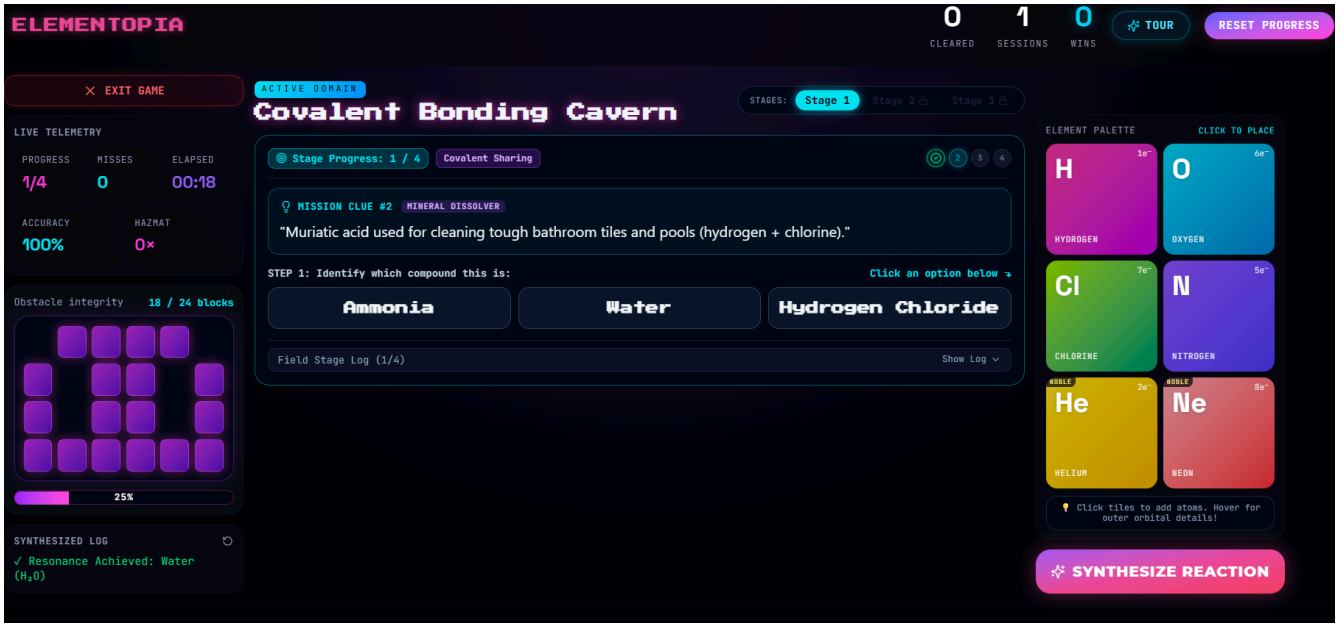
Elementopia 1.4: Learning Room Progression ERD



### *3.1.5 Random Compound Generation Per Session*

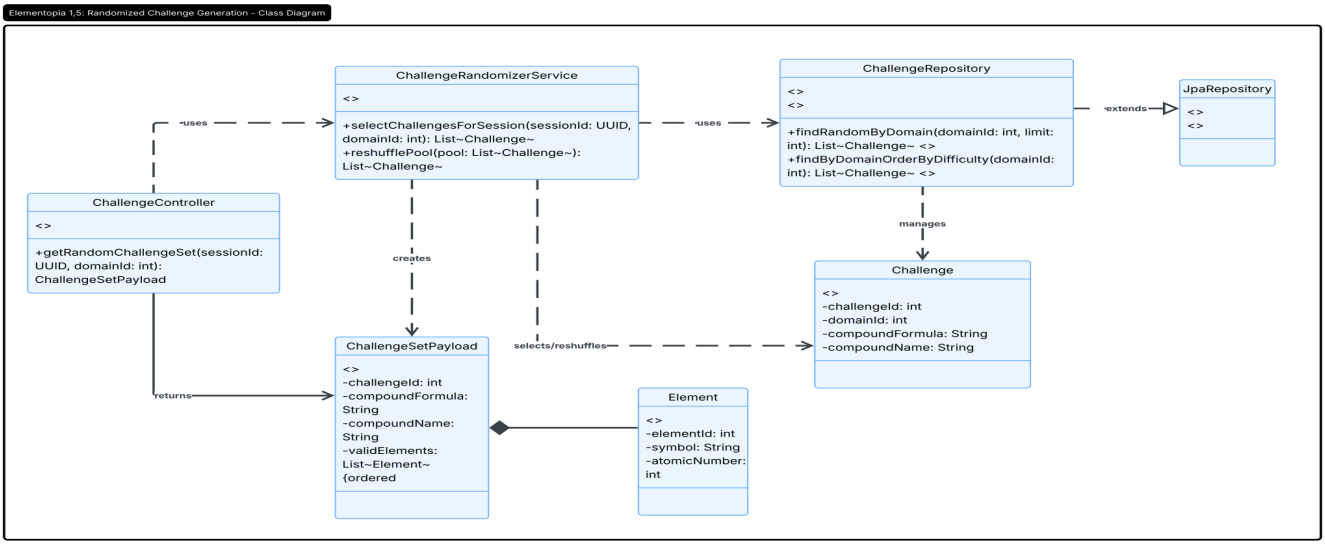
- User Interface Design



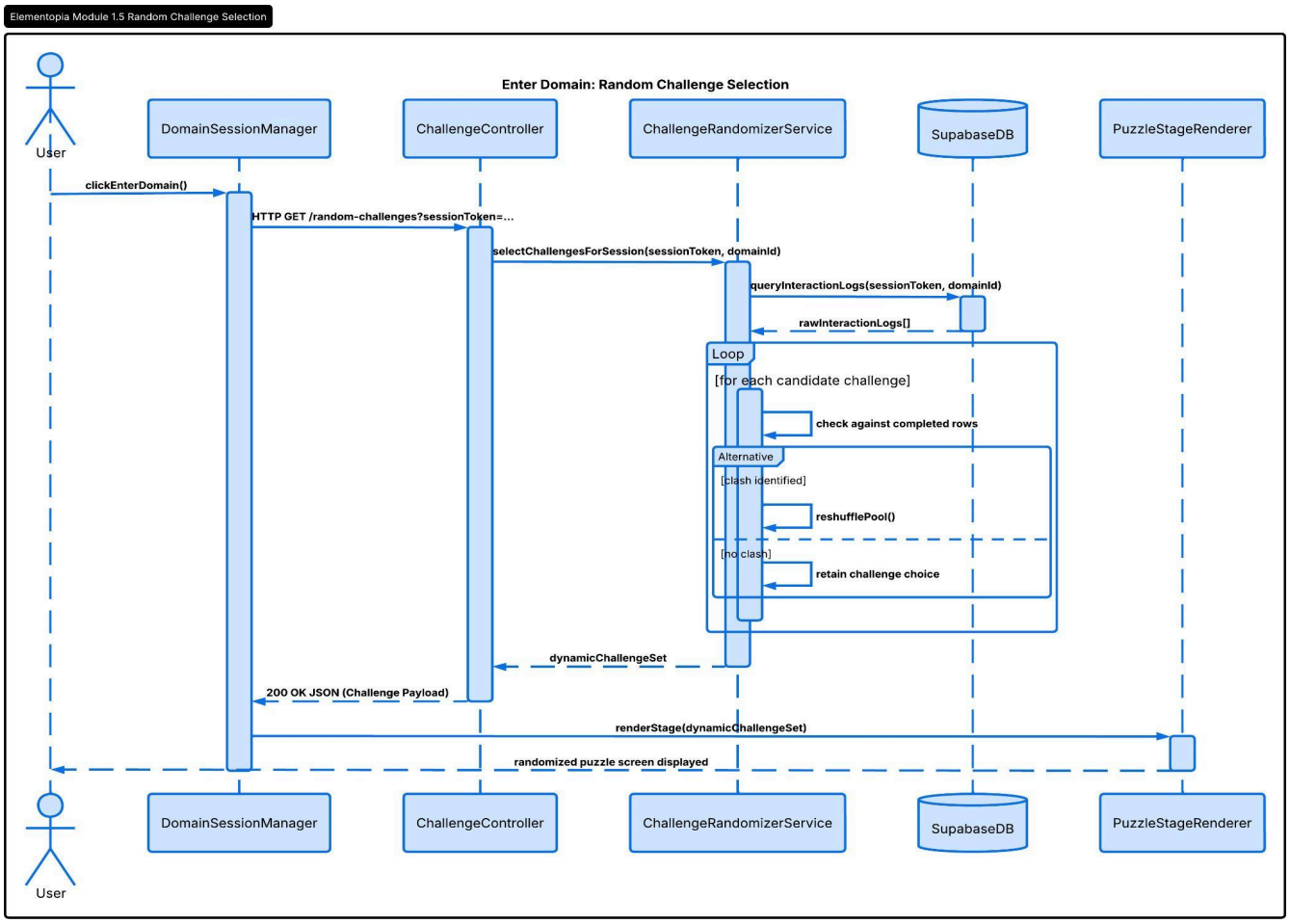




- Front-end component(s)
  - **Component name:** DomainSessionManager
    - **Description and Purpose:** This acts as the container or page-level component that wraps a self-paced puzzle room session. When a student enters a Domain, this component initializes the session state, requests the randomized, non-repeated challenge set from the backend, and handles client-side state caching for the session's duration.

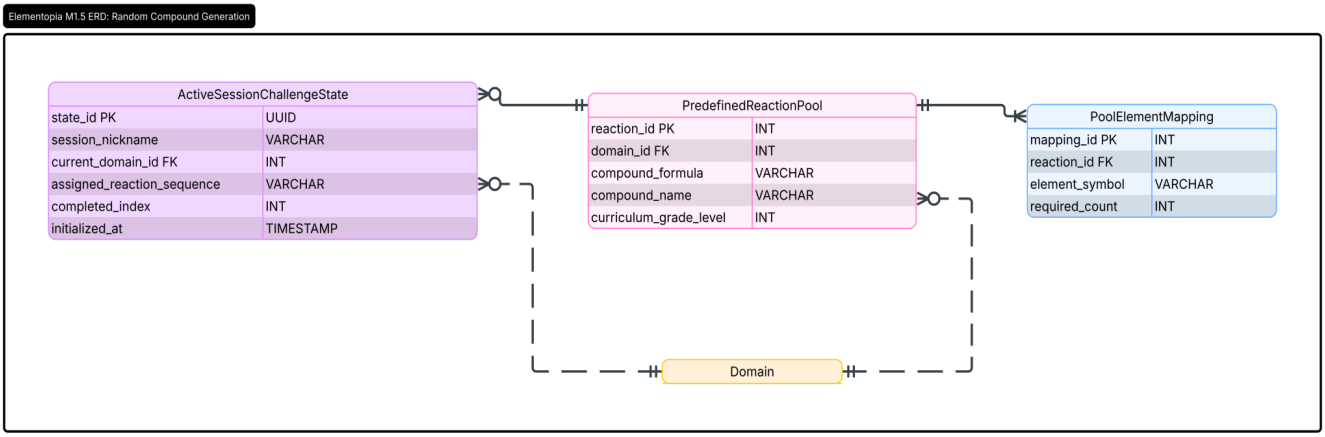
        - **Component Type or Format:** React Functional Page Component (.jsx / .tsx).
    - **Component name:** PuzzleStageRenderer
        - **Description and Purpose:** Responsible for reading the randomized challenge configuration data passed from the manager and dynamically painting the user interface. It calculates the positions of the dynamic element layouts and updates the background environmental obstacle visuals based on the selected challenge.
        - **Component Type or Format:** React Functional Presentation Component (.jsx / .tsx).
- Back-end component(s)
    - **Component name: ChallengeController**
        - **Description and Purpose:** Exposes the REST API endpoint that the front-end requests when a user clicks into a Domain level. It receives the student's current session identifier and target Domain ID, coordinating with service layers to return the payload.
        - **Component Type or Format:** Spring Boot RestController Java Class.
    - **Component name: ChallengeRandomizerService**
        - **Description and Purpose:** Contains the central algorithmic selection logic. It fetches the standard pool of high school chemistry reactions for the specified domain from the database, checks the user's history to find previously completed milestones in that current session, runs a reshuffling algorithm to discard duplicates, and selects a fresh, unrepeated challenge set.
        - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components
    - Class Diagram:



- Sequence Diagram




- Data Design
  - ERD or schema



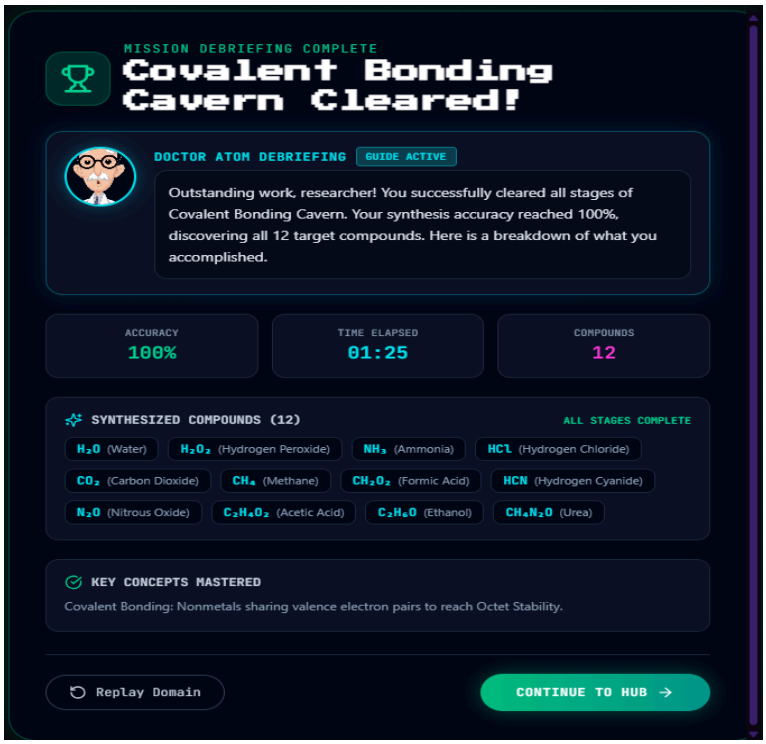
### *3.1.6 Post-Synthesis Educational Debrief*

- **User Interface**



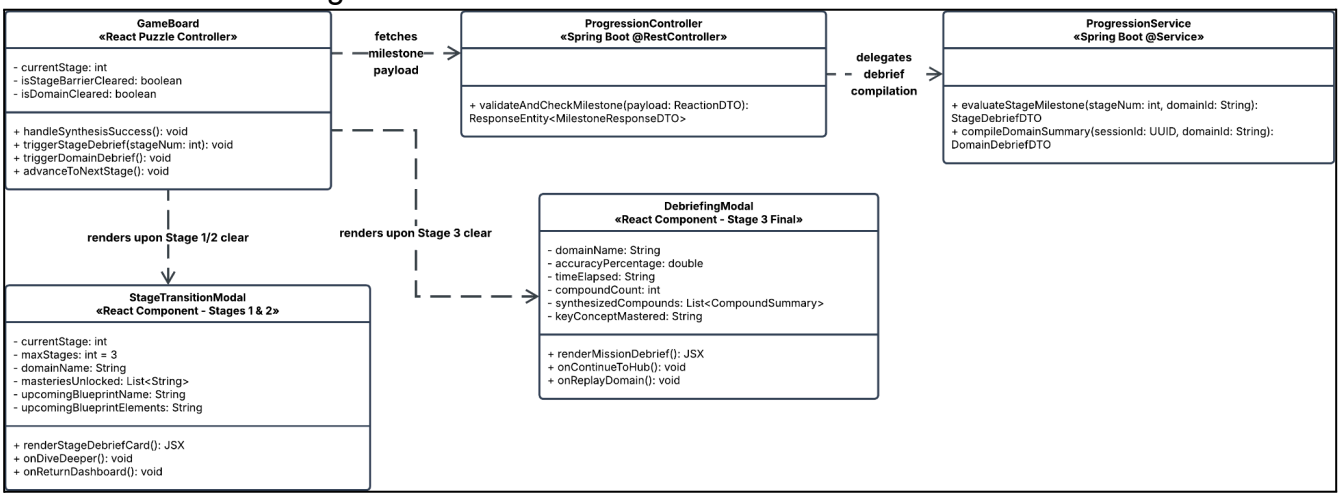




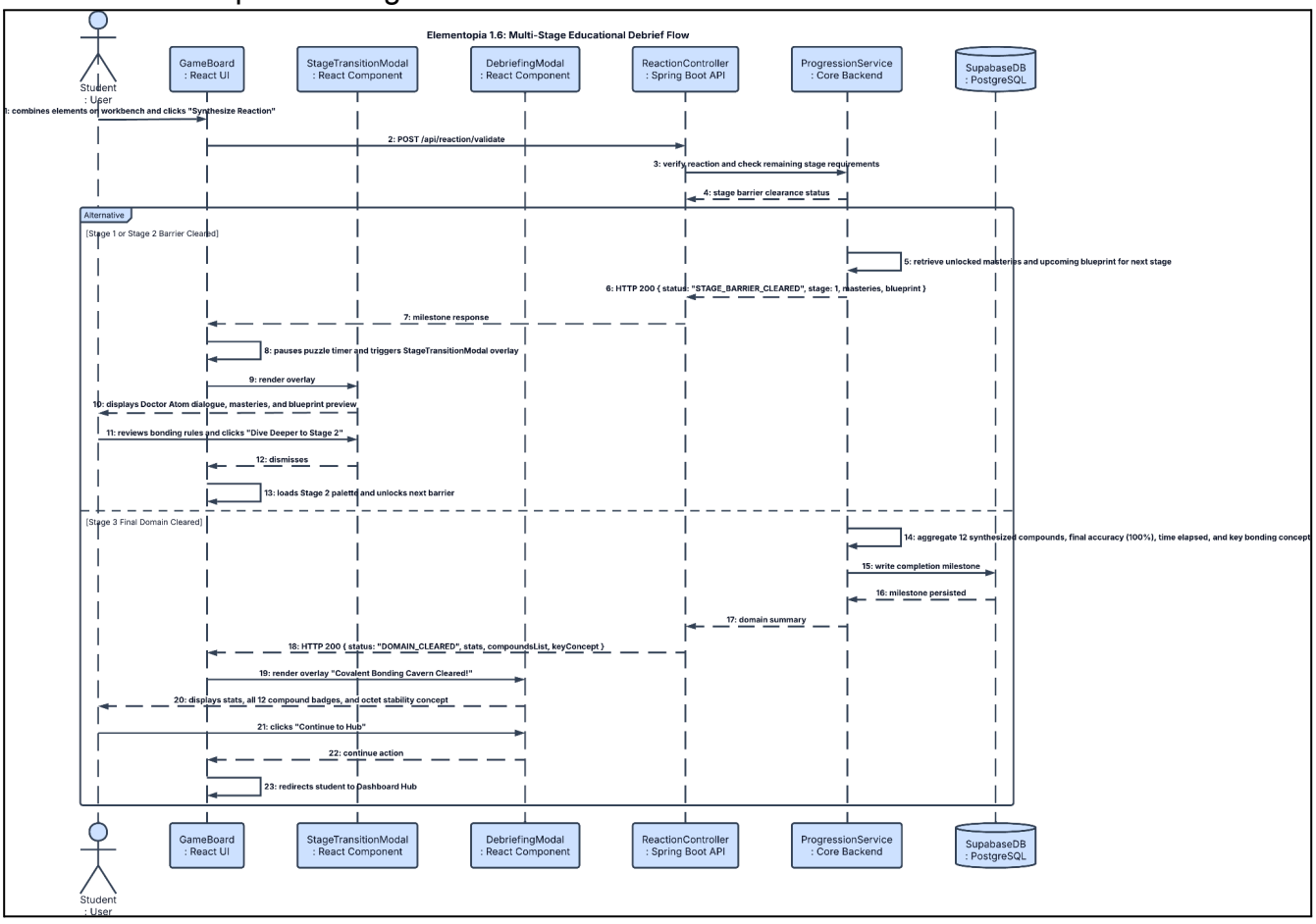


    - Displays two specialized modal overlays at key pedagogical milestones:
        - **Stage Debriefing Overlay:** Renders *“Stage {X} Barrier Cleared!”* featuring Doctor Atom's commentary, a checklist of unlocked bonding masteries, and an upcoming target blueprint card with a prominent navigation button (*“Dive Deeper to Stage {X+1}”*).
        - **Domain Completion Overlay:** Renders *“{Domain Name} Cleared!”* featuring Doctor Atom’s mission evaluation, performance metric cards (Accuracy %, Time Elapsed, Total Compounds), a grid of all 12 synthesized compound badges with formulas, and a summary of key bonding concepts mastered.
- Frontend Component(s):
    - **Component Name:** StageTransitionModal.jsx
        - **Description and Purpose:** Intercepts the clearance of intermediate stage barriers (Stages 1 and 2). It pauses canvas interactions, presents stage-specific bonding masteries, and smoothly transitions the inventory tray and obstacle layout to the next stage upon user confirmation.
        - **Component type or format:** React.js functional modal component with animated backdrop blur.
    - **Component Name:** DebriefingModal.jsx
        - **Description and Purpose:** Renders upon the completion of all three stages in a domain. It aggregates session gameplay telemetry, displays the complete 12-compound discovery list with formulas, reinforces the governing bonding theory, and routes the student back to the Hub or allows replaying the domain.
        - **Component type or format:** React.js comprehensive modal component.
- Backend Component(s):
    - **Component Name:** ReactionController.java
        - **Description and Purpose:** Validates the final stage reaction, confirms barrier dissolution conditions, and passes milestone achievement indicators to the frontend client.
        - **Component type or format:** Spring Boot @RestController.
    - **Component Name:** ProgressionService.java
        - **Description and Purpose:** Tracks stage-by-stage unlock states within domain.stages, calculates cumulative accuracy and elapsed time, and supplies the upcoming stage blueprint metadata.
        - **Component type or format:** Spring Boot @Service component.

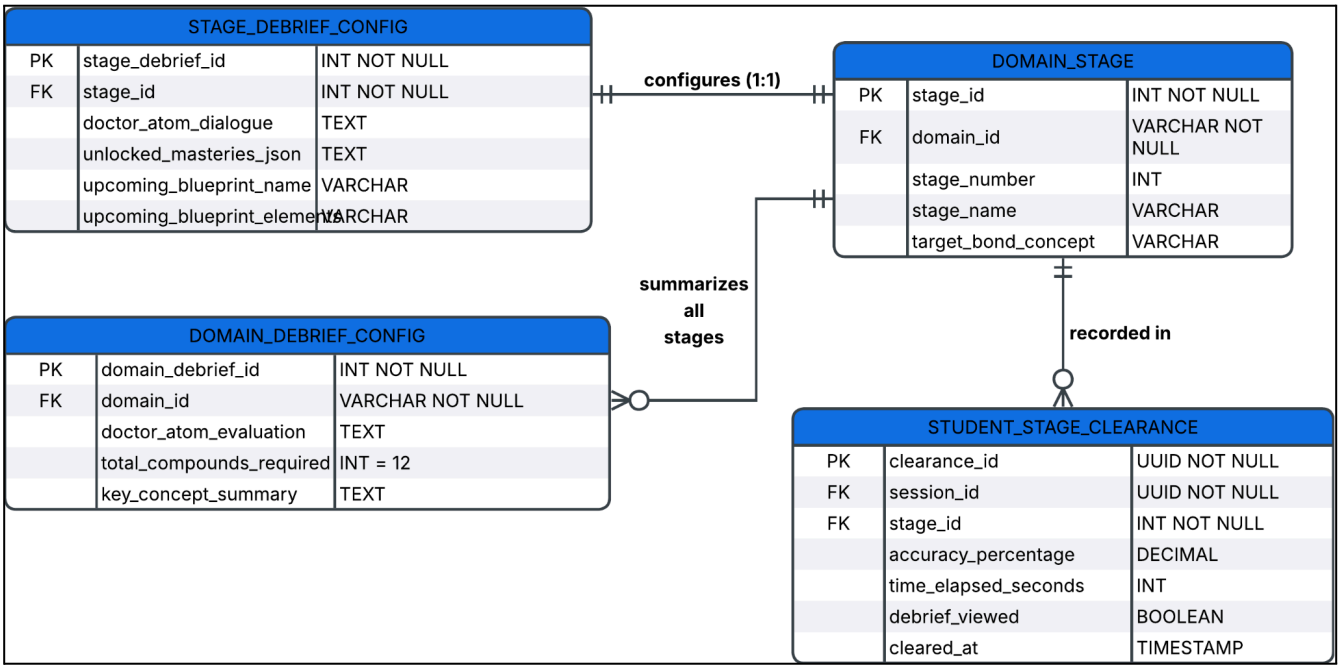
- Object-Oriented Components
  - Class Diagram




  - Sequence Diagram



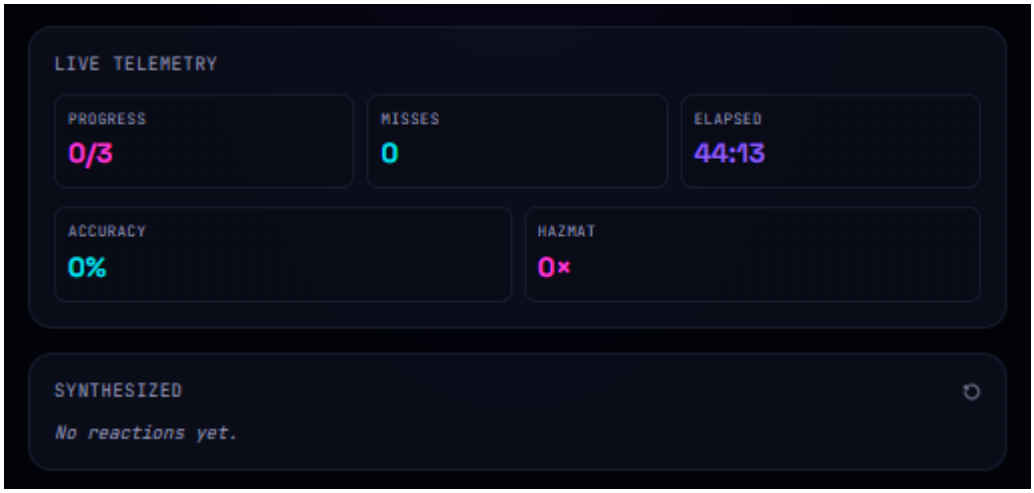
- Data Design
    - ERD or schema



## *3.2. Module 2: Assessment & Progress Module*

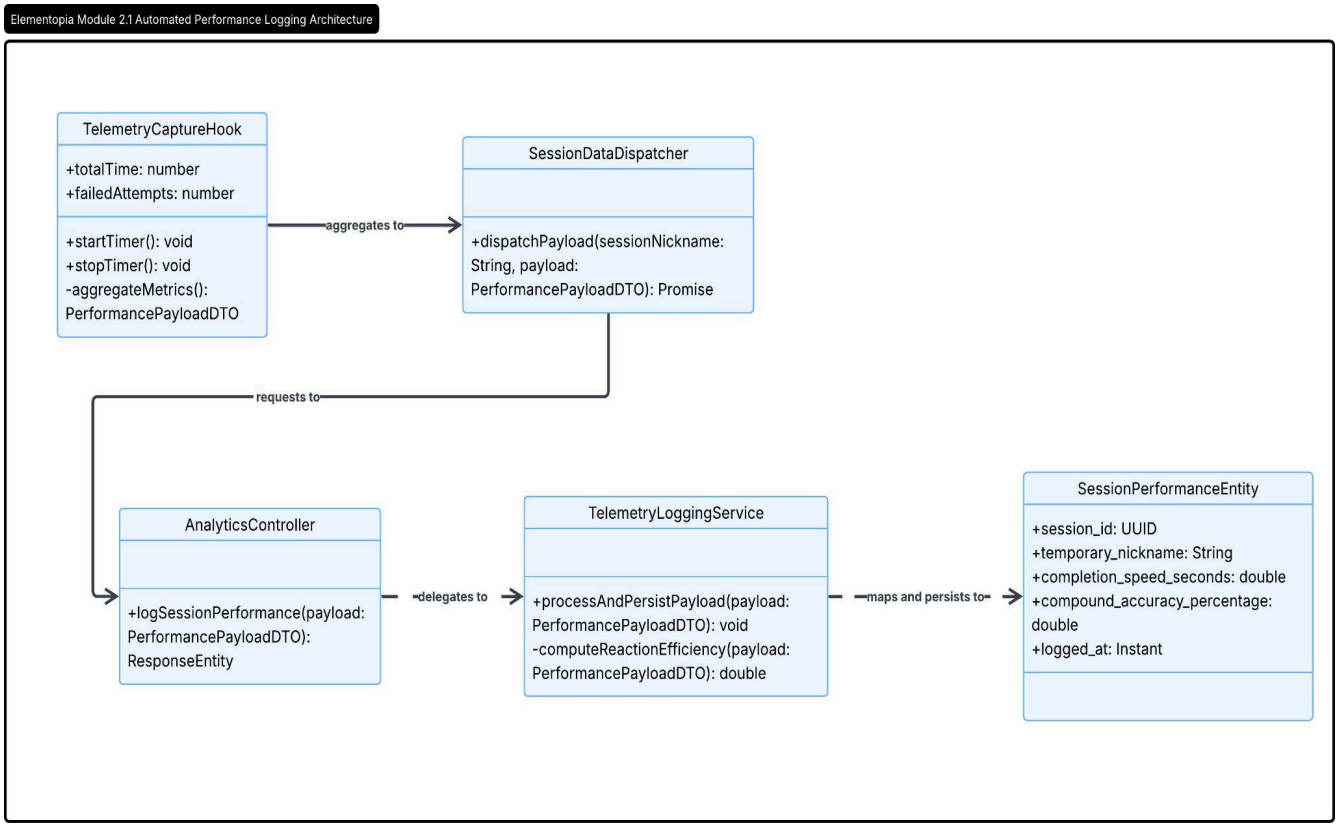
### *3.2.1 Automated Session Performance Logging*

- User Interface Design

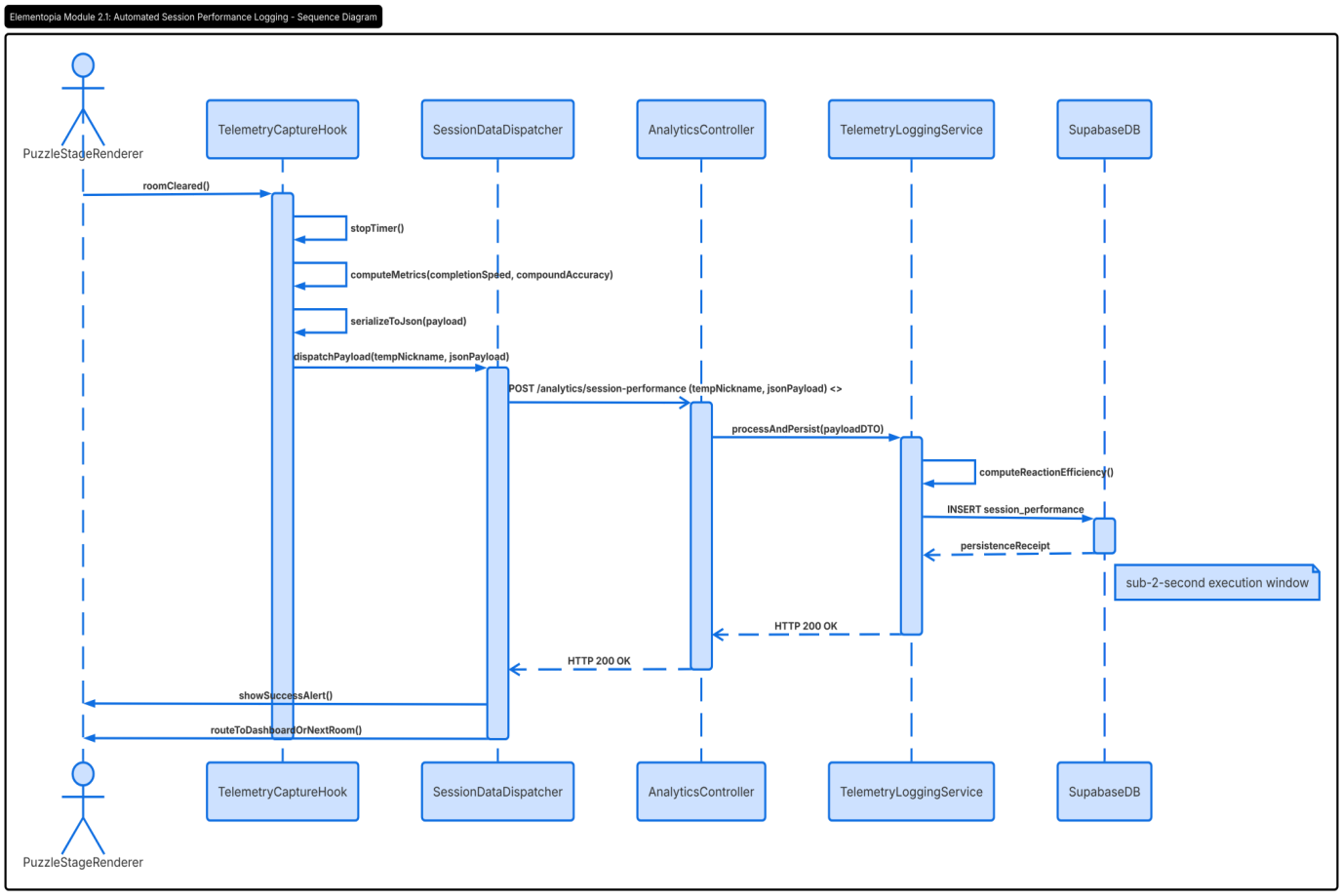



- Front-end component(s)
  - **TelemetryCaptureHook**
    - **Description and Purpose:** A custom hook that monitors active gameplay duration and tracks interaction counters (such as valid syntheses and failed attempts) during a puzzle session. Upon detecting a cleared or completed state, it automatically stops timers, aggregates the behavioral metrics into a performance data transfer object (DTO), and prepares the transactional payload for cloud delivery via the persistence lifecycle layer.
    - **Component Type or Format:** React Custom Hook (.js / .ts).
  - **SessionDataDispatcher**
    - **Description and Purpose:** Manages the network lifecycle for persisting data. It takes the payload generated by the telemetry hook, serializes it, and sends an asynchronous transmission to the backend API. It enforces user-friendly error boundaries to prevent application crashes if network anomalies occur, and signals the user interface to transition once a write confirmation is verified.
    - **Component Type or Format:** React Service / API Utility Layer Module.
- Back-end component(s)
  - **AnalyticsController**
    - **Description and Purpose:** Exposes secure REST endpoints (e.g., /api/analytics/session/log) to receive raw performance logs from the client application. It maps inbound payloads to structured request data

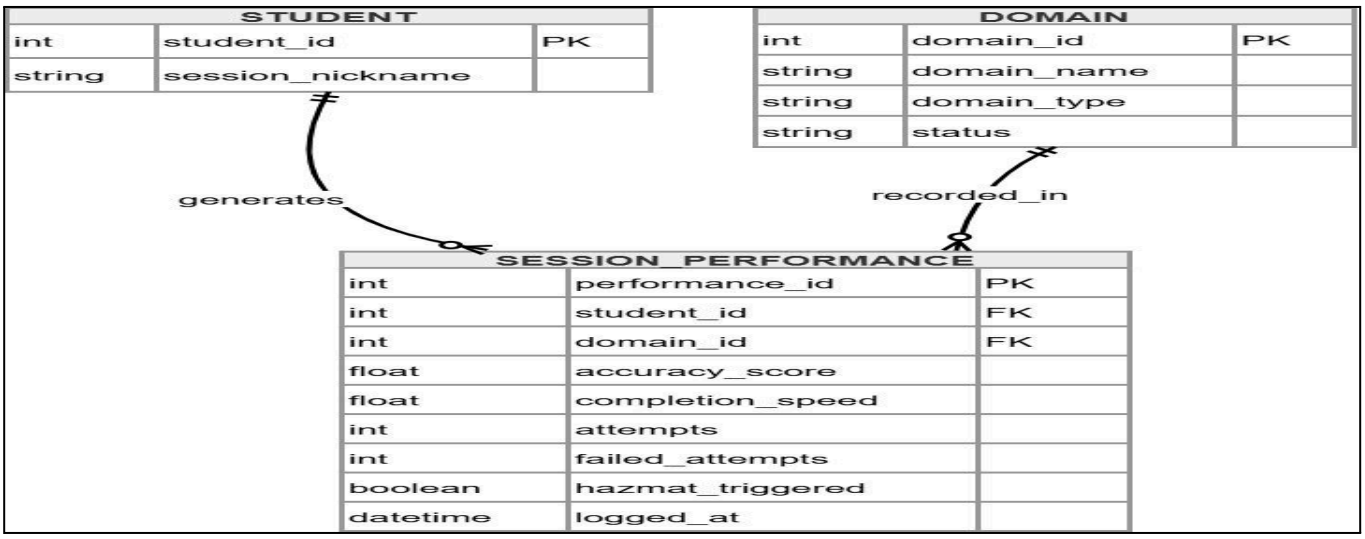
transfer objects and hands them off to the tracking service layer for processing.
    - **Component Type or Format:** Spring Boot RestController Java Class.
  - **TelemetryLoggingService**
    - **Description and Purpose:** Houses the core analytical and database synchronization logic. It receives structural DTOs, performs validation on performance path efficiency computations, associates data records with the zero-registration temporary session nickname, and orchestrates the database write operations to Supabase within the mandated sub-2-second constraint.
    - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components
  - Class Diagram



- Sequence Diagram

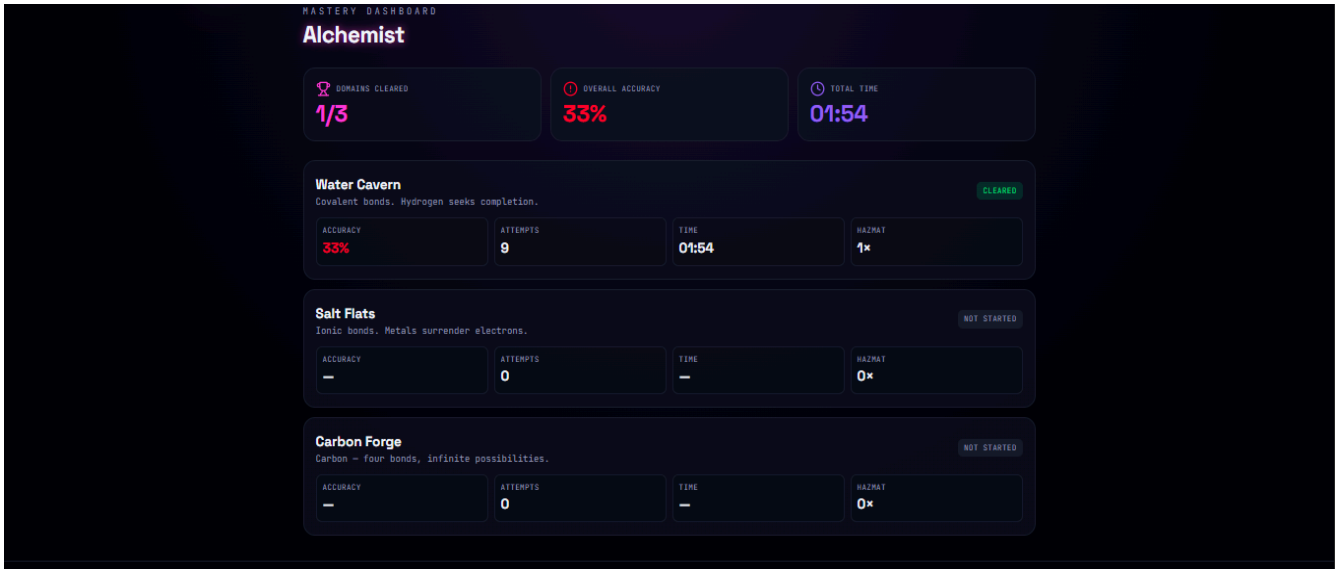



- Data Design
  - ERD or schema



### *3.2.2 Mastery Dashboard Visualization*

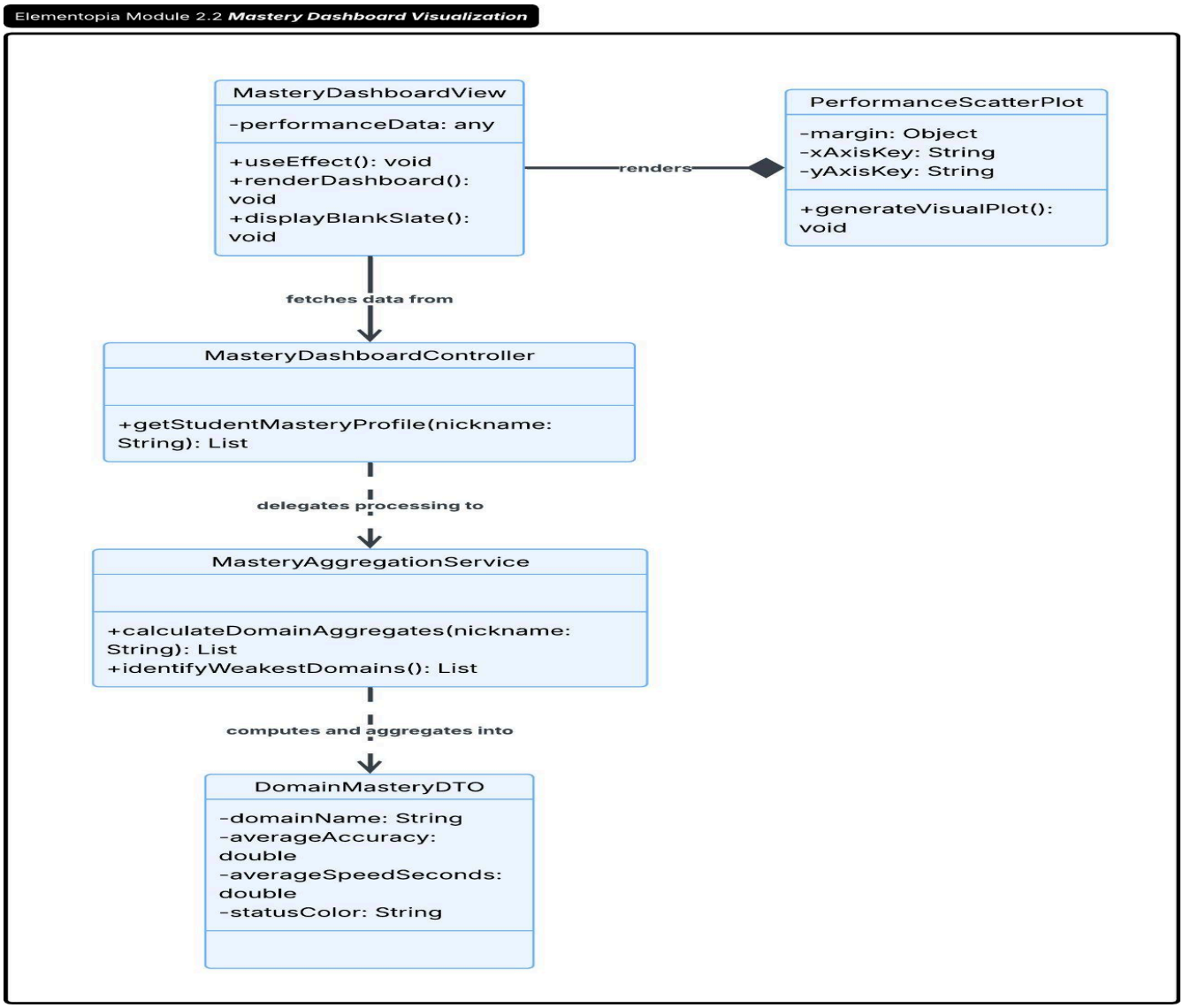
- User Interface Design




- Front-end component(s)
  - **MasteryDashboardView**
    - **Description and Purpose:** Acts as the primary page container for rendering student performance history. Upon loading, it fetches historical session telemetry from Supabase. If no historical metrics exist, it renders a "Blank Slate" UI prompting the student to join their first level.
    - **Component Type or Format:** React Functional Page Component (.jsx / .tsx).
  - **PerformanceScatterPlot**
    - **Description and Purpose:** A data visualization component that maps historical metrics dynamically. It plots completion speed against compound accuracy per Domain, automatically highlighting weak areas (e.g., rendering low-accuracy sections in red) so students can independently target conceptual gaps.
    - **Component Type or Format:** React Presentation Component utilizing an external charting library (e.g., Recharts or Chart.js).
- Back-end component(s)
  - **MasteryDashboardController**
    - **Description and Purpose:** Exposes the REST API endpoint (e.g., /api/mastery/{nickname}) to retrieve consolidated data profiles for a student based on their temporary session nickname.
    - **Component Type or Format:** Spring Boot RestController Java Class.
  - **MasteryAggregationService**

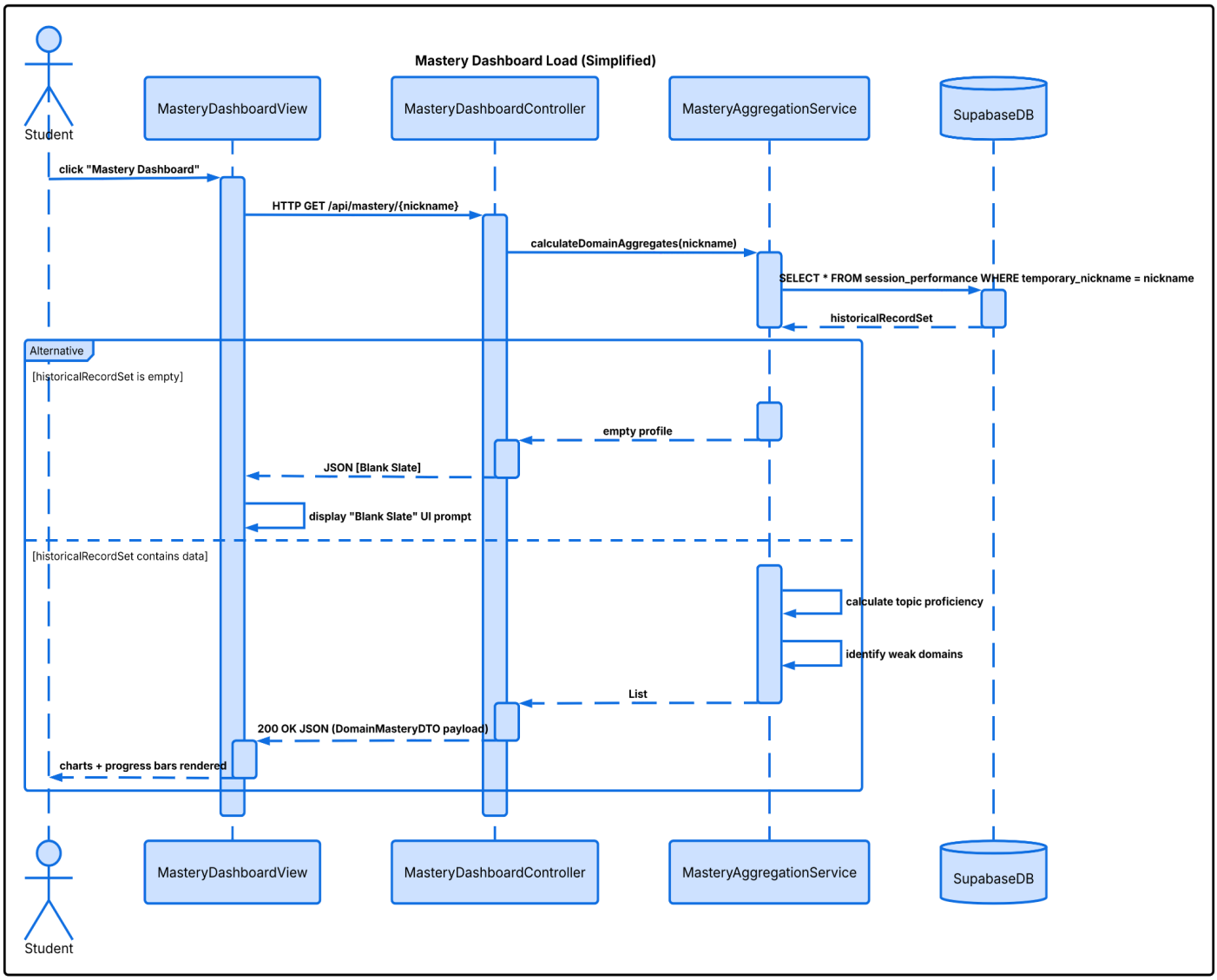
- **Description and Purpose:** Handles the core business and analytics logic. It queries raw performance logs from the database, computes aggregate scores for compound accuracy and average completion velocity per Domain, and returns a tailored structural metrics payload optimized for frontend rendering.
- **Component Type or Format:** Spring Boot @Service Java Class.

- Object-Oriented Components
  - Class Diagram

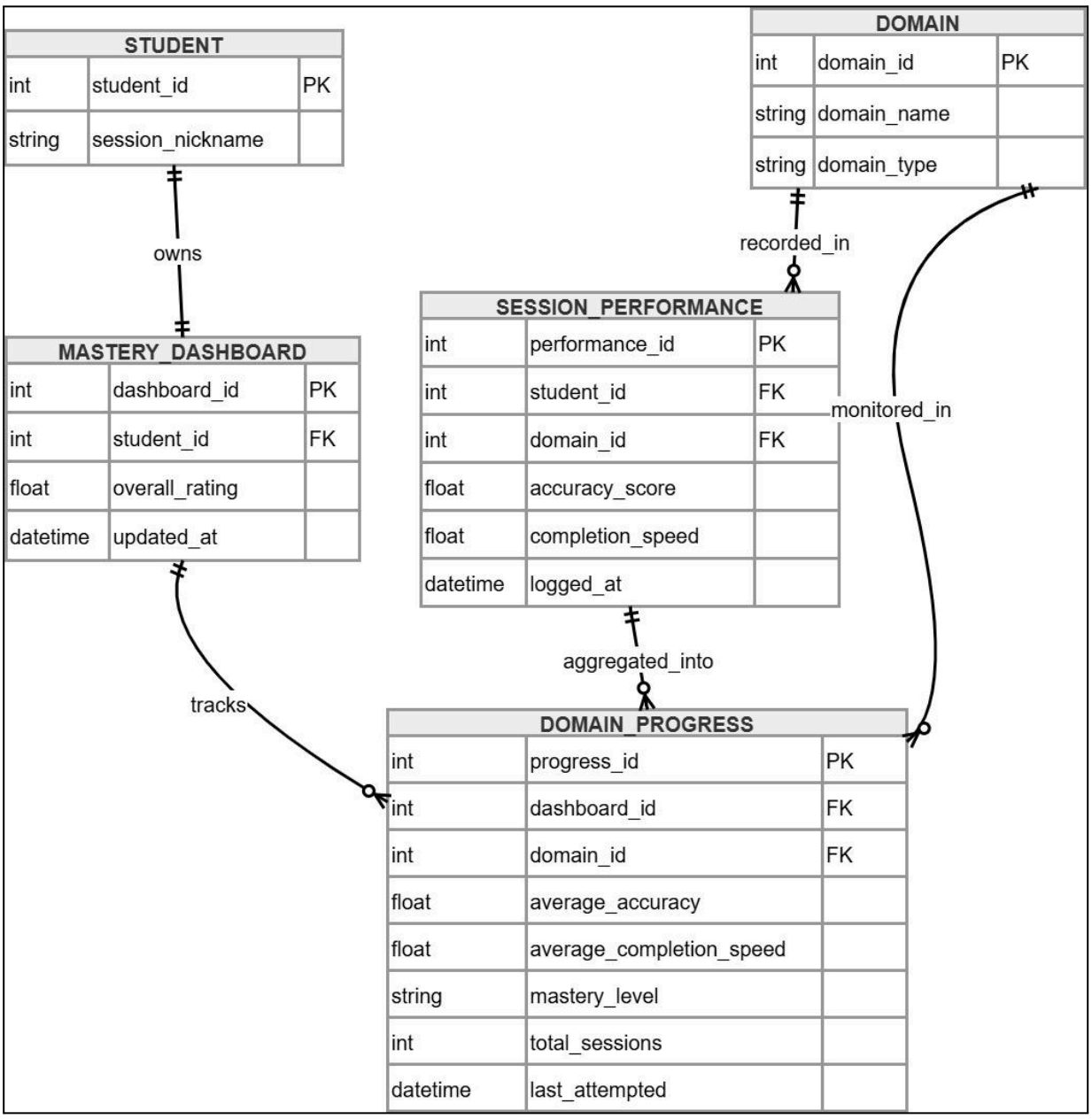



  - Sequence Diagram

Elementopia Module 2.2 Mastery Dashboard Load

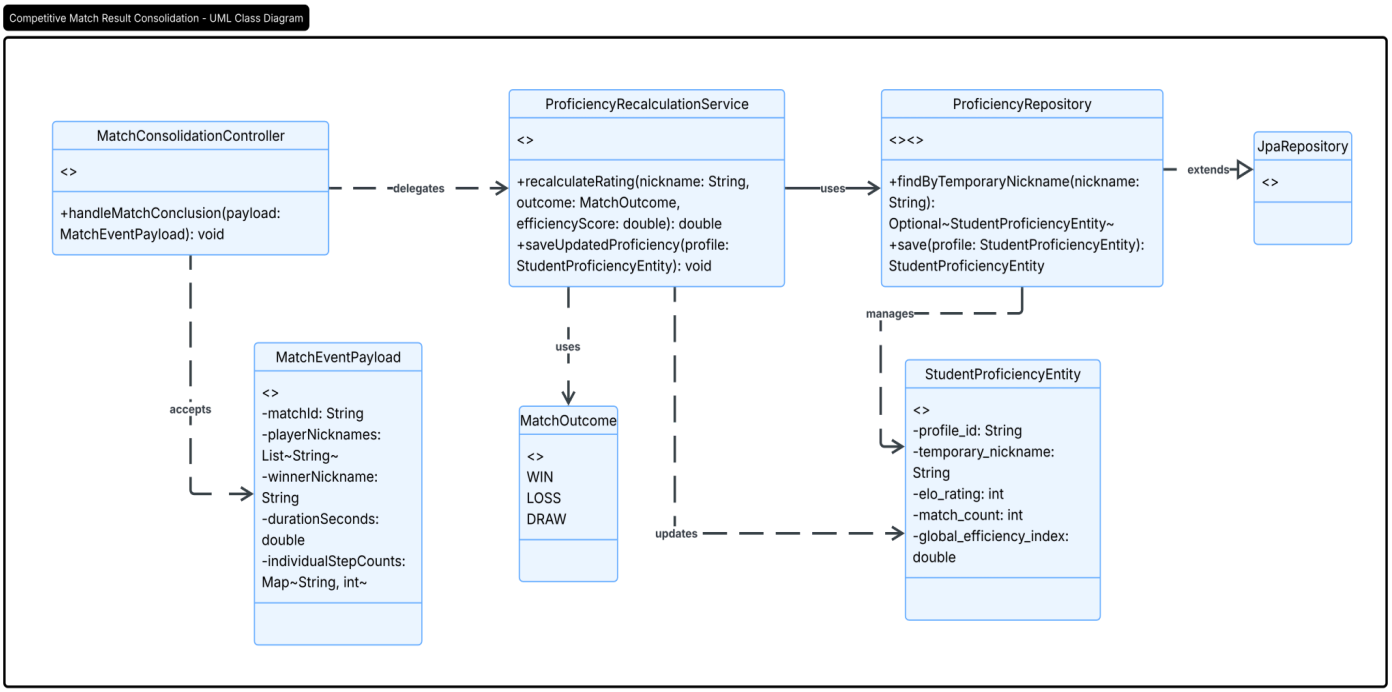


- Data Design
    - ERD or schema



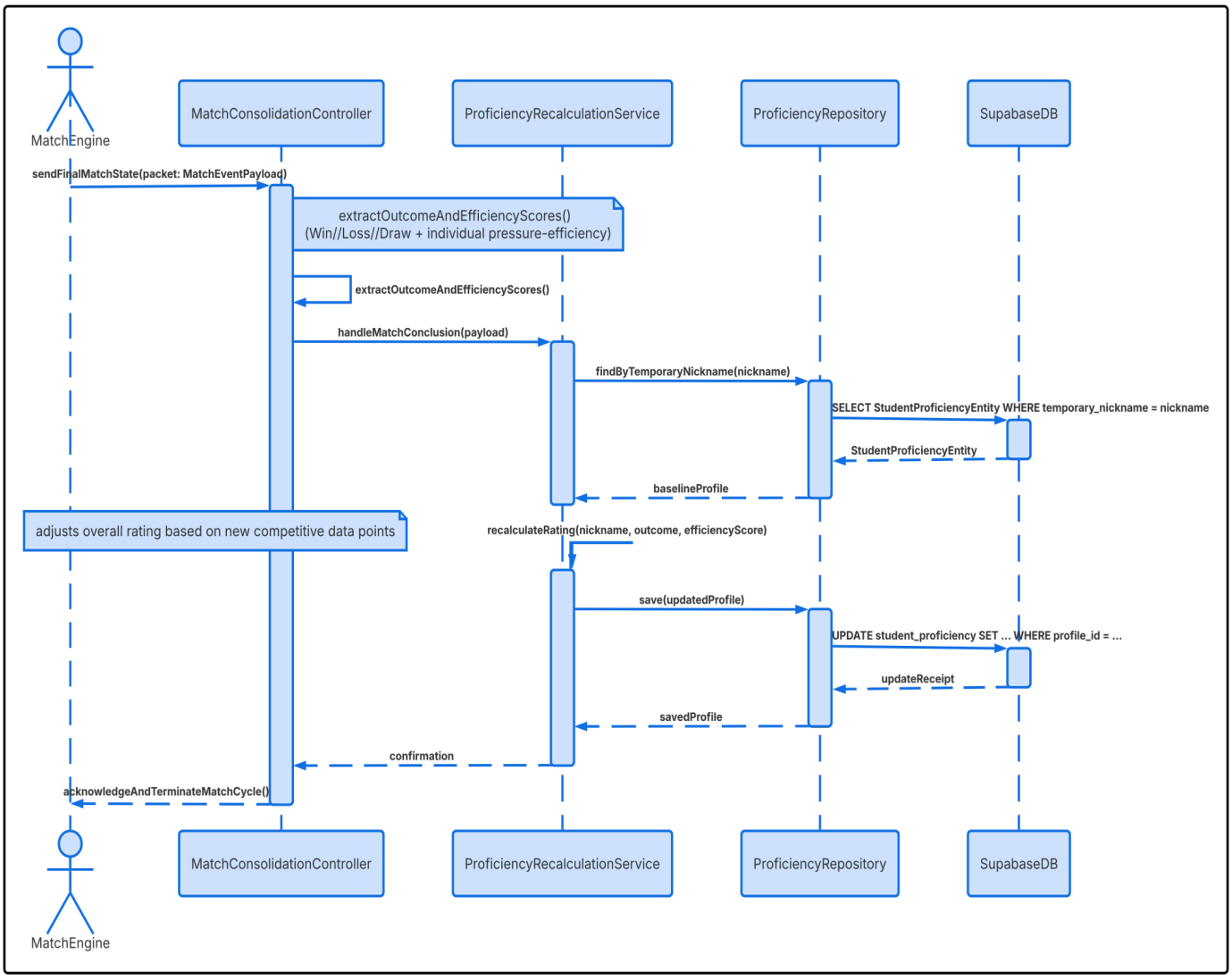
### *3.2.3 Competitive Match Result Consolidation*

- Back-end component(s)
    - **MatchConsolidationController**
        - **Description and Purpose:** Exposes internal endpoints or acts as the listener boundary that handles incoming match conclusion events from the multiplayer game loop. It extracts the raw match final state (Win/Loss/Draw) along with the player's pressure-efficiency scores and directs them to the consolidation workflow.
        - **Component Type or Format:** Spring Boot RestController / Event Listener Java Class.
    - **ProficiencyRecalculationService**
        - **Description and Purpose:** Houses the core algorithmic rating formulas. It retrieves the student's existing proficiency profile from Supabase, processes the new match results alongside their Speed-to-Compound efficiency metrics, computes the adjusted rating, and pushes the updated data record back to persistent storage.
        - **Component Type or Format:** Spring Boot @Service Java Class.
- Object-Oriented Components
    - Class Diagram

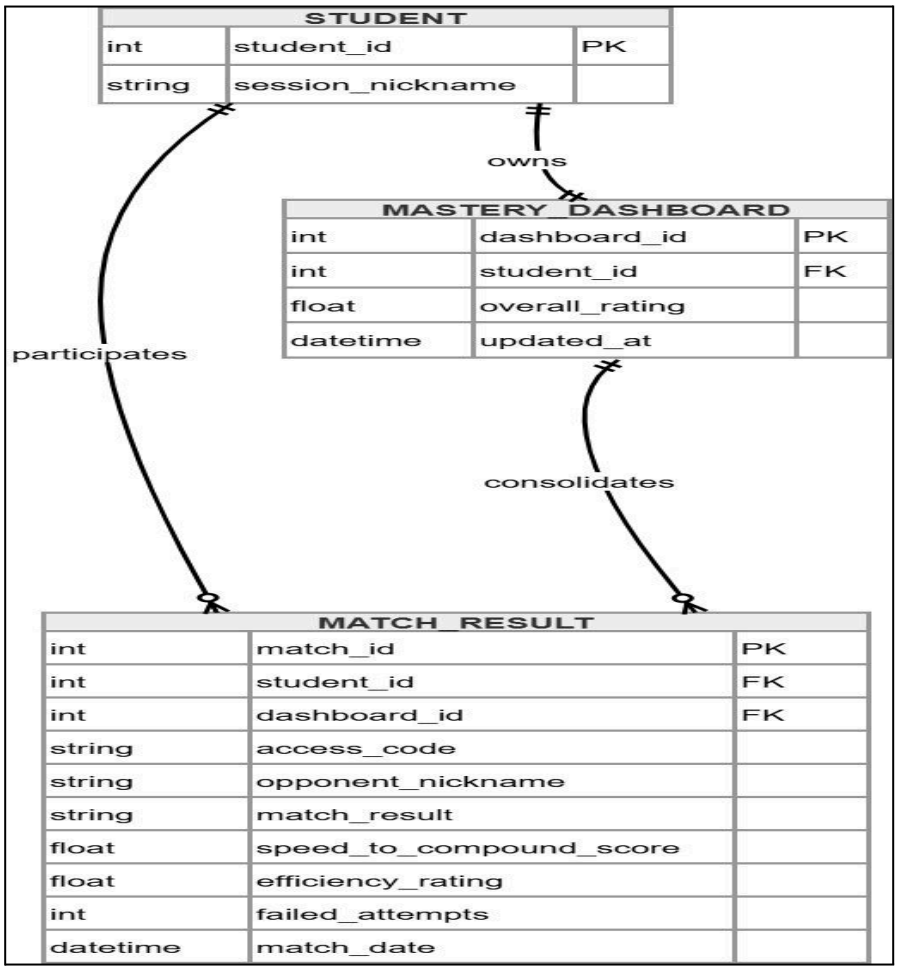



    - Sequence Diagram

Elementopia M2.3: Competitive Match Result - Sequence Diagram

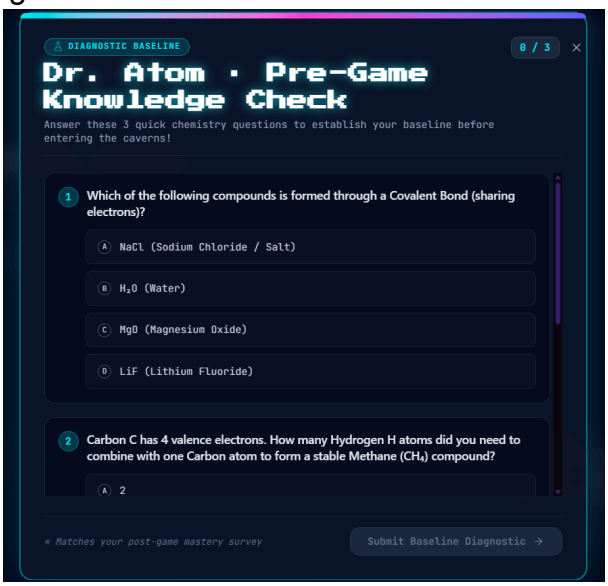


- Data Design
    - ERD or schema



### *3.2.4 Centralized Diagnostic Pre-Test Cloud Sync*

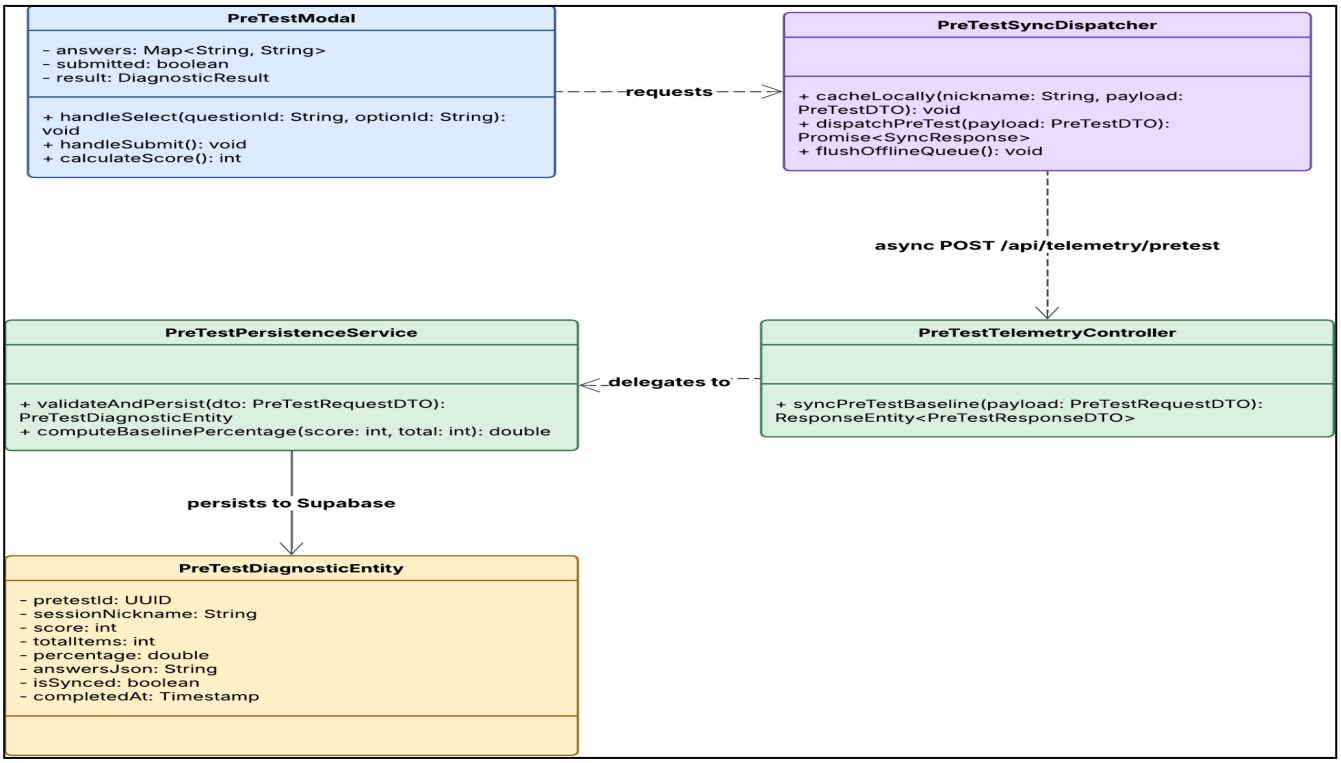
- User Interface Design:



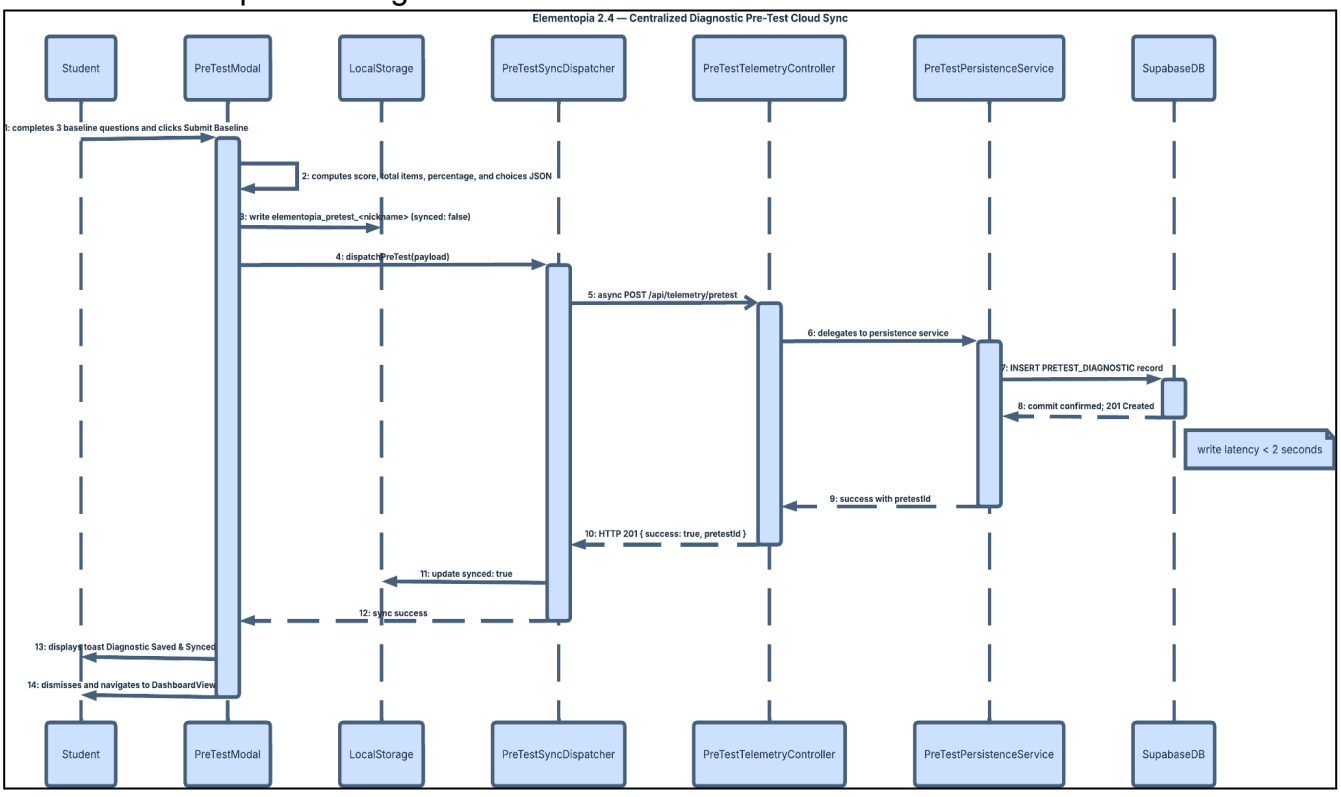

- Front-end component(s):
  - Component Name: PreTestModal.jsx
    - Description and Purpose: Presents the diagnostic questions, computes baseline percentage, caches the result locally in localStorage (synced = false), and triggers background cloud dispatch.
  - Component type or format: React.js interactive modal component.
  - Component Name: PreTestSyncDispatcher.js
    - Description and Purpose: Manages asynchronous REST transmission to /api/telemetry/pretest. Upon receiving HTTP 201, updates local storage to synced = true. If the network is offline, queues the payload for auto-flush on reconnect.
  - Component type or format: JavaScript API service module.
- Back-end component(s):
  - Component Name: PreTestTelemetryController.java
    - Description and Purpose: Exposes POST /api/telemetry/pretest to ingest student baseline scores and serialized choice selections.
  - Component type or format: Spring Boot @RestController.
  - Component Name: PreTestPersistenceService.java
    - Description and Purpose: Commits diagnostic telemetry records to Supabase in under 2 seconds, linking the record to the session UUID.
  - Component type or format: Spring Boot @Service class.

- Object-Oriented Components

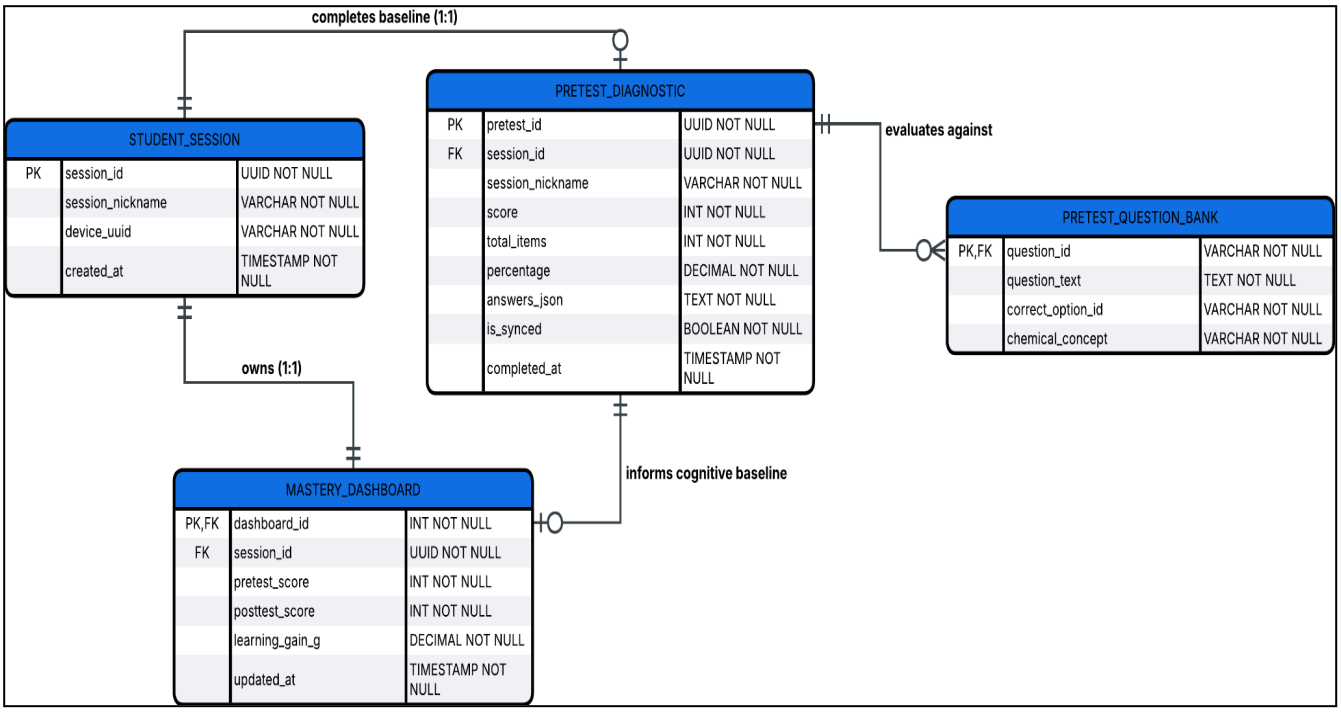
- Class Diagram




- Sequence Diagram



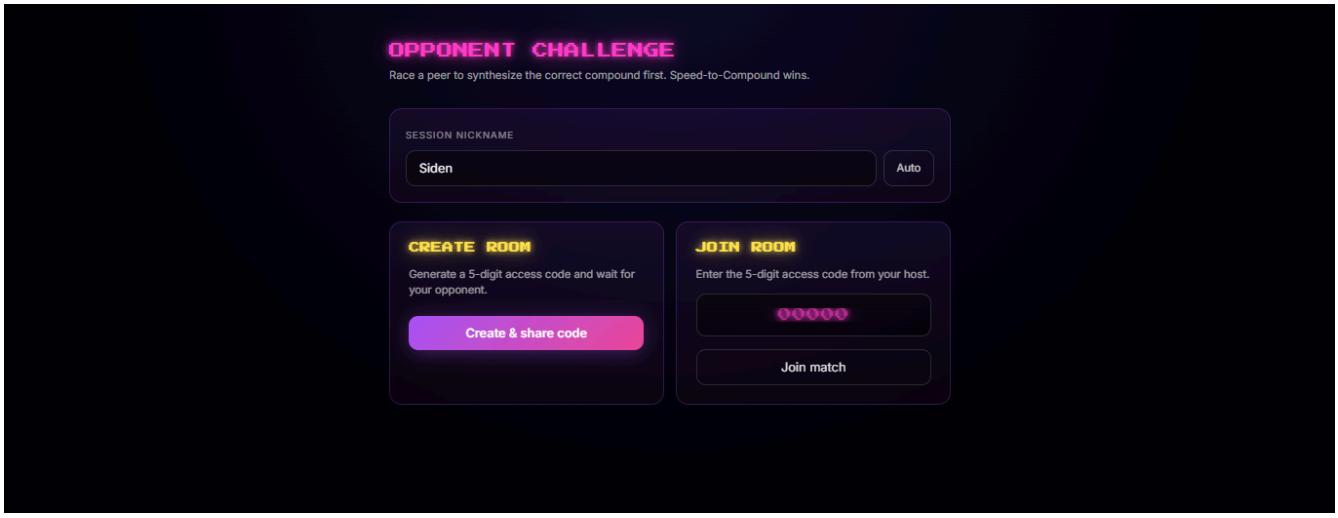
- Data Design
    - ERD or schema



## *3.3. Module 3: Opponent-Based Challenge Module*

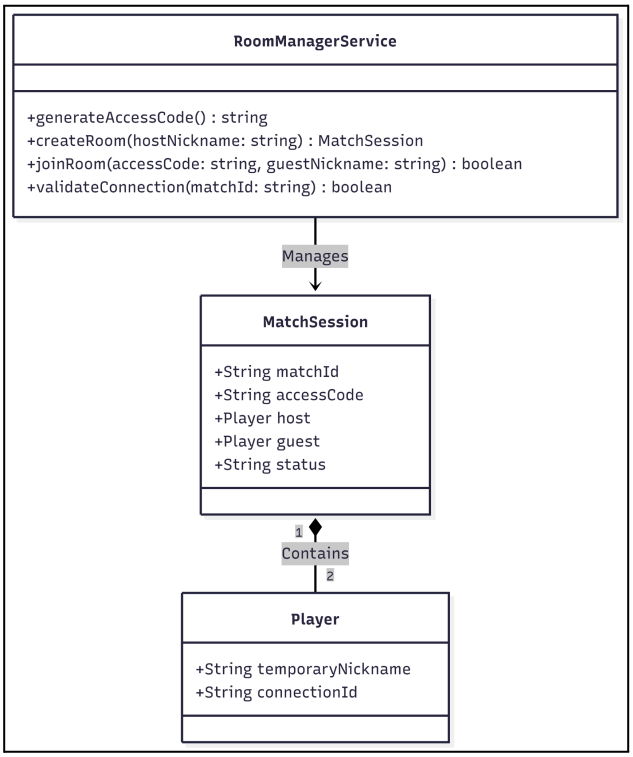
### *3.3.1 Room Creation and Matchmaking*

- User Interface Design

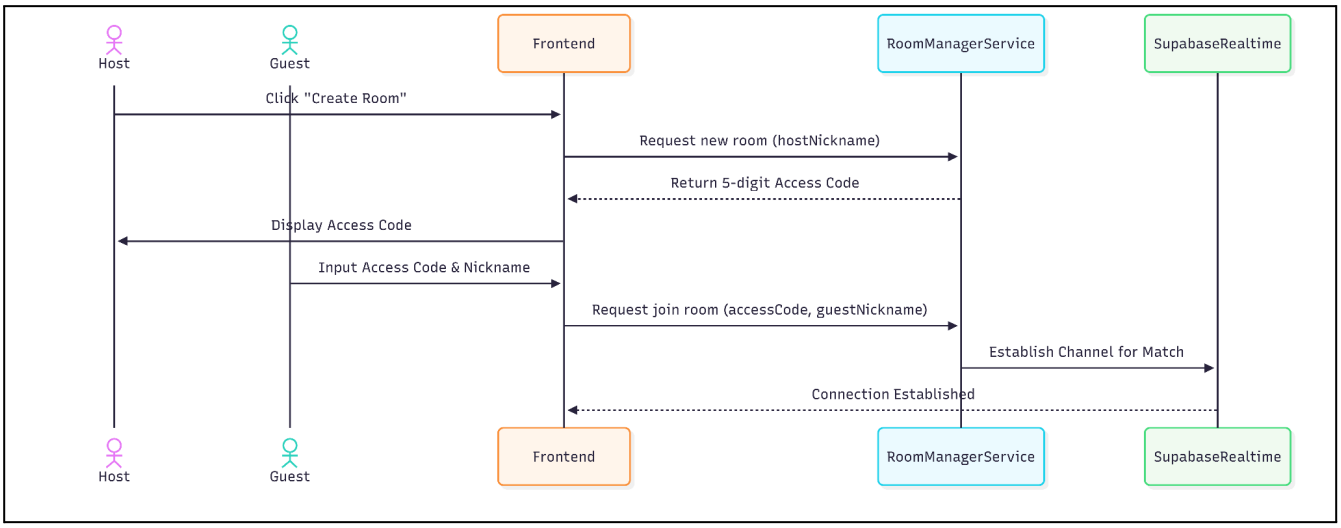



- Front-end component(s)
  - MatchmakingController
    - Manages UI state for room creation and joining, displaying auto-generated codes or input forms.
    - React Functional Component
  - CodeInputModal
    - Captures the 5-digit access code and the guest's temporary session nickname.
    - React Modal Component
- Back-end component(s)
  - RoomManagerService
    - Handles generation of the 5-digit code, registers the room session in memory, and establishes WebSocket connections via Supabase Realtime.
    - Spring Boot Service Class
- Object-Oriented Components

- Class Diagram




- Sequence Diagram

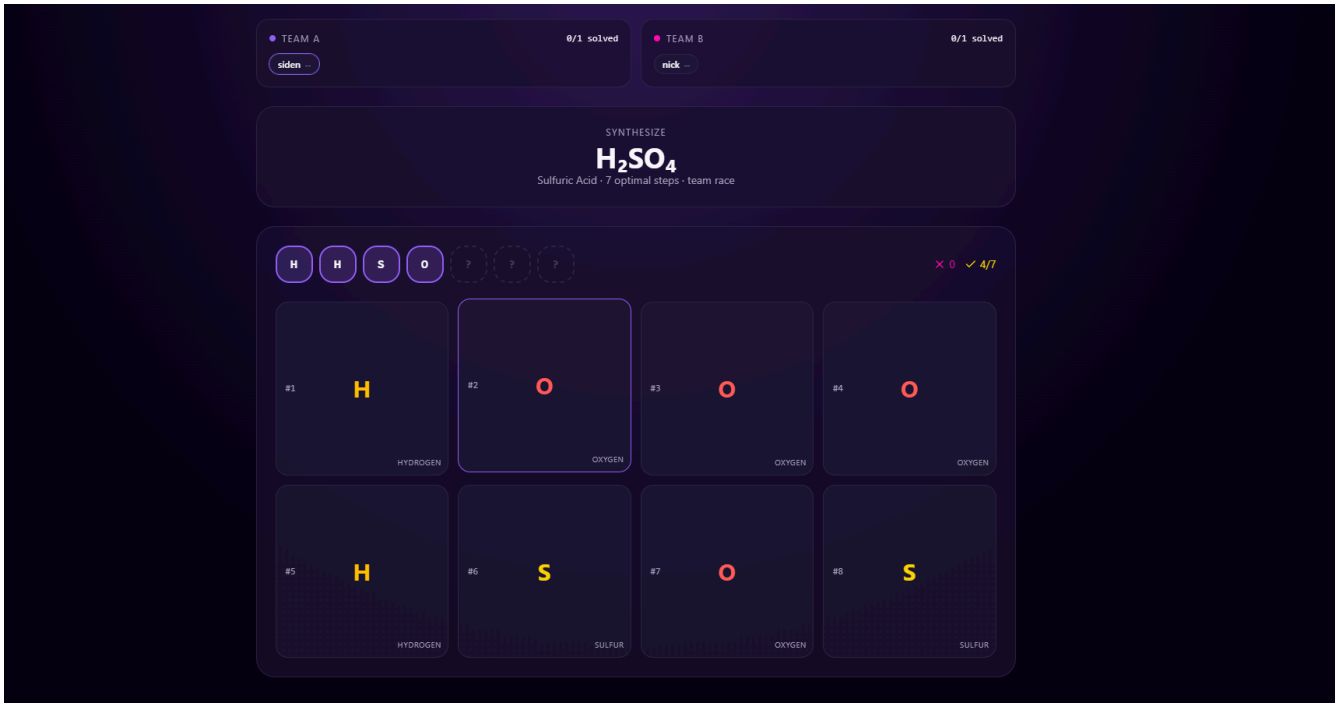



- Data Design
  - ERD or schema

| ACTIVE_MATCH_LOBBY | | | |
|---|---|---|---|
| string | matchId | PK | Unique UUID for the match lobby |
| string | hostNickname | | Session name of creator |
| string | guestNickname | | Session name of joiner |
| string | accessCode | | Auto-generated 5-digit join code |
| string | status | | WAITING, IN_PROGRESS, COMPLETED |
| timestamp | createdAt | | Time the room was created |

### *3.3.2 Synchronized Competitive Task Generation*

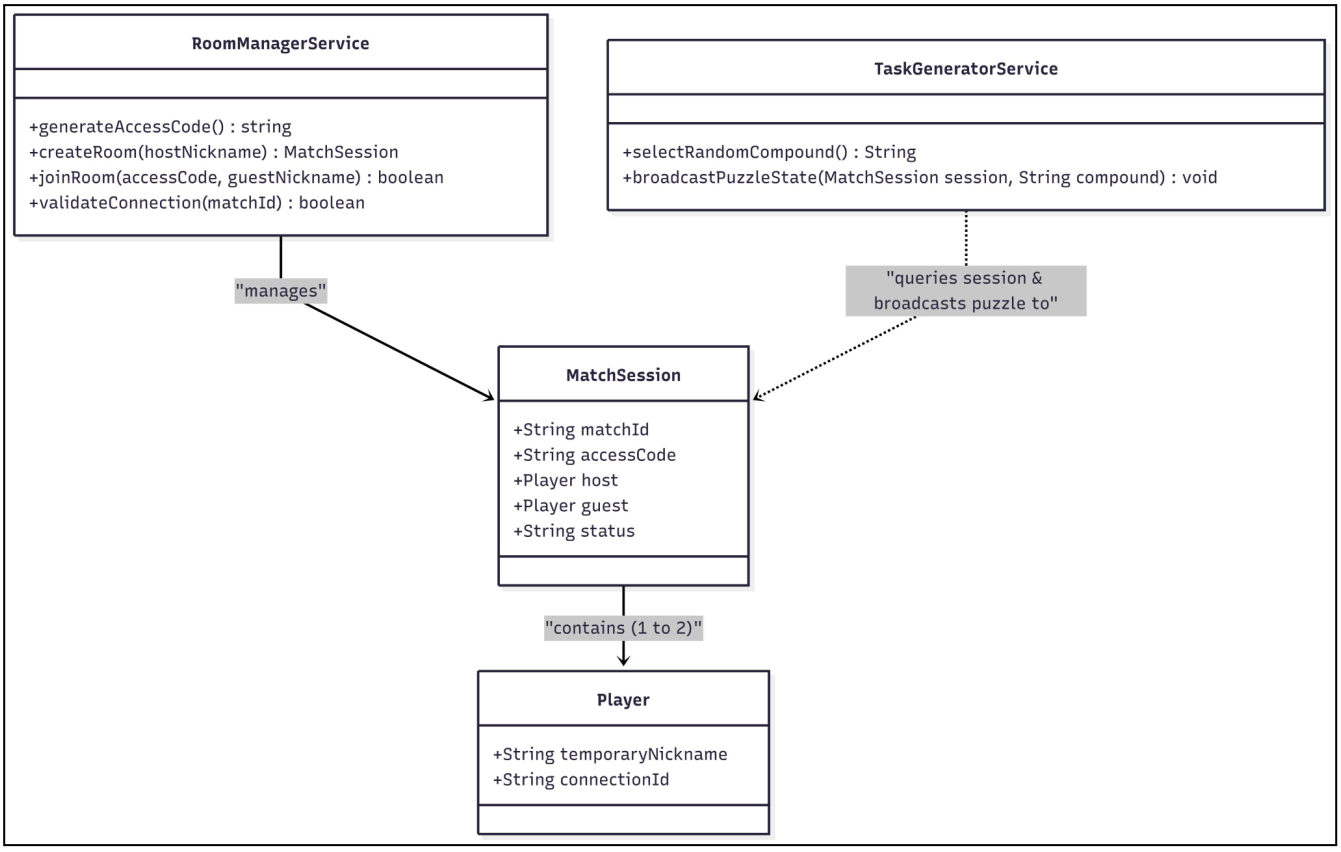
- User Interface Design



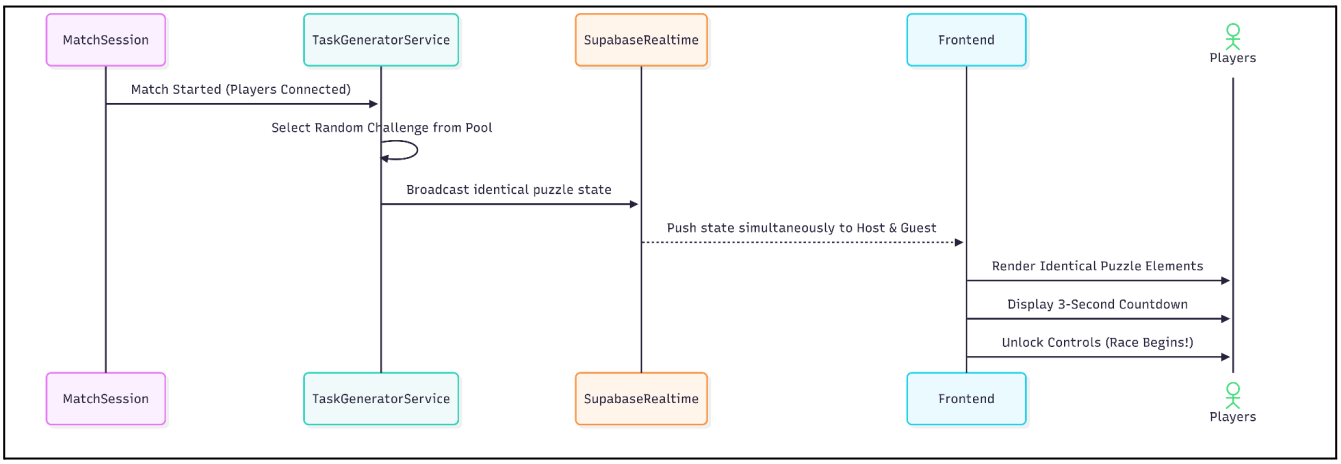

- Front-end component(s)

- CompetitivePuzzleBoard
  - Renders the environmental obstacle and element inventory. Locks controls during the initial countdown to ensure a fair start.
  - React Functional Component
- CountdownOverlay
  - Displays the synchronized 3-second visual countdown before unlocking player controls
  - React Component / Animation hook

- Back-end component(s)
  - TaskGeneratorService
    - Selects a randomized compound challenge from the predefined pool and broadcasts it to both clients to ensure 100% identical scenarios.
    - Spring Boot Service Class
- Object-Oriented Components
  - Class Diagram



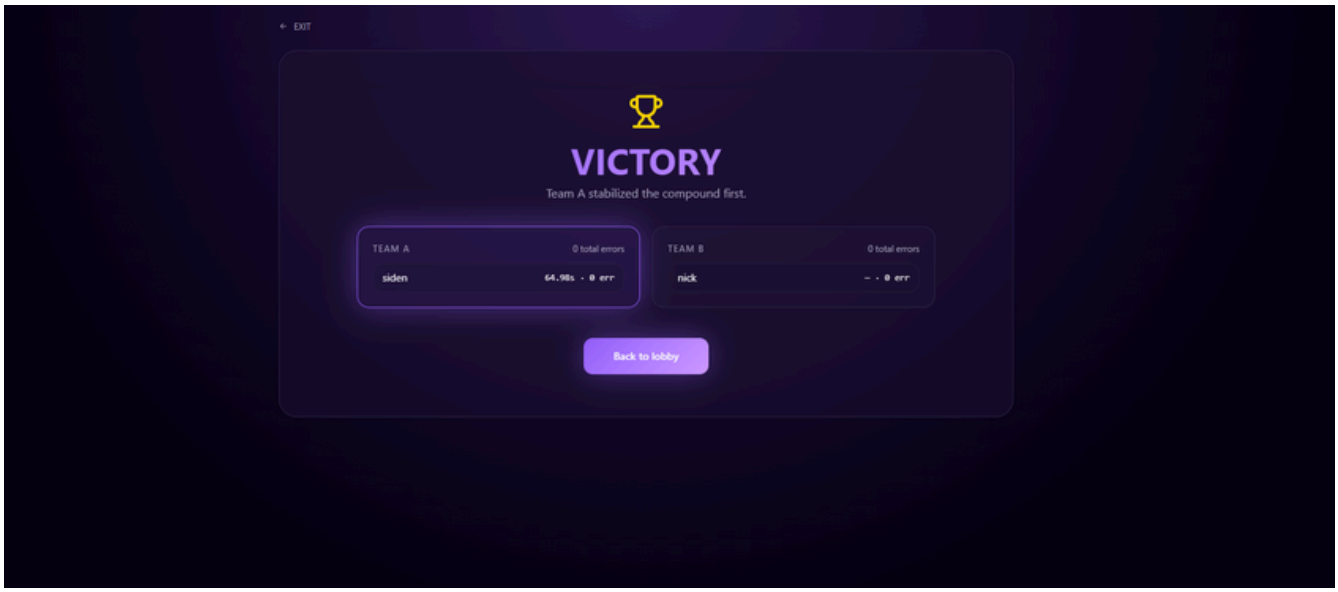

- Sequence Diagram




- Data Design
  - ERD or schema

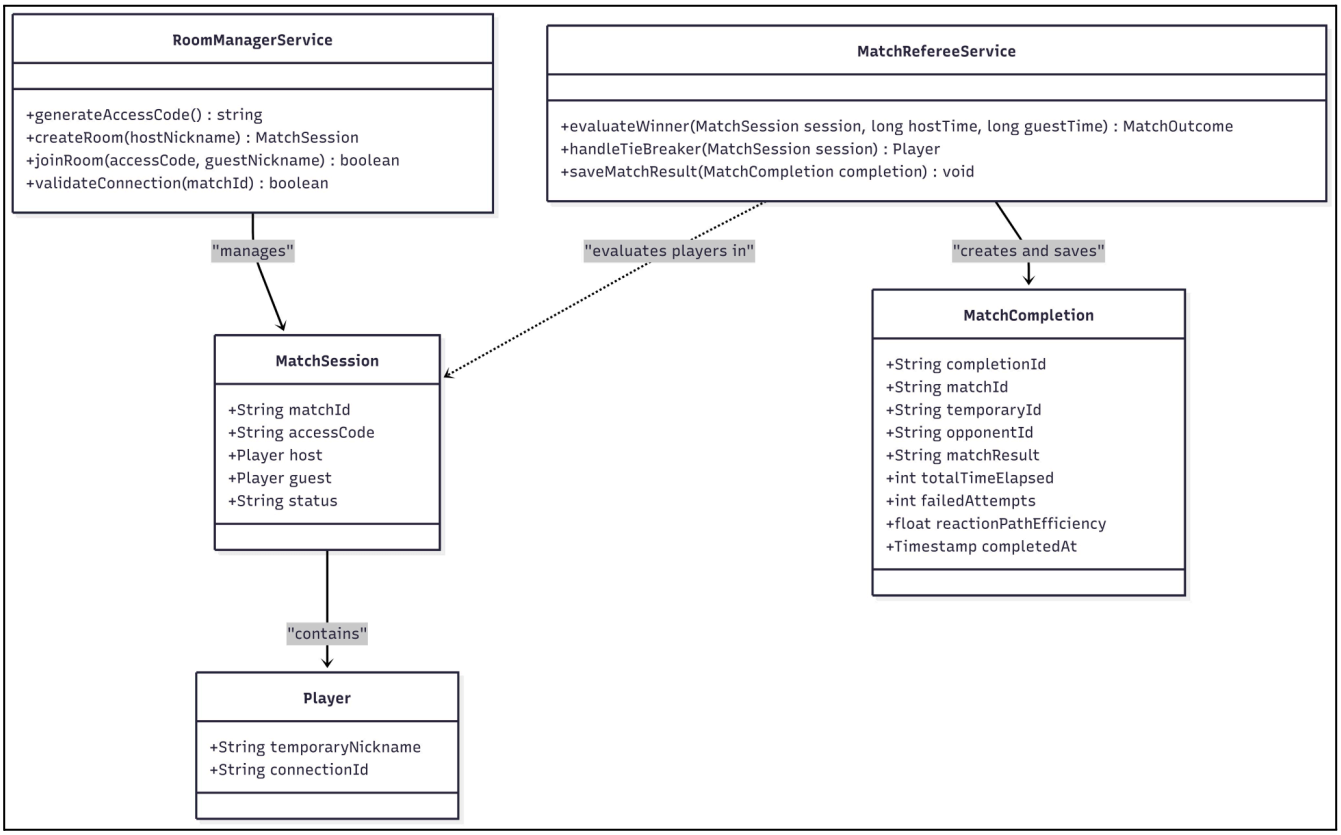
### *3.3.3 Real-Time Speed-to-Compound Evaluation*

- User Interface Design

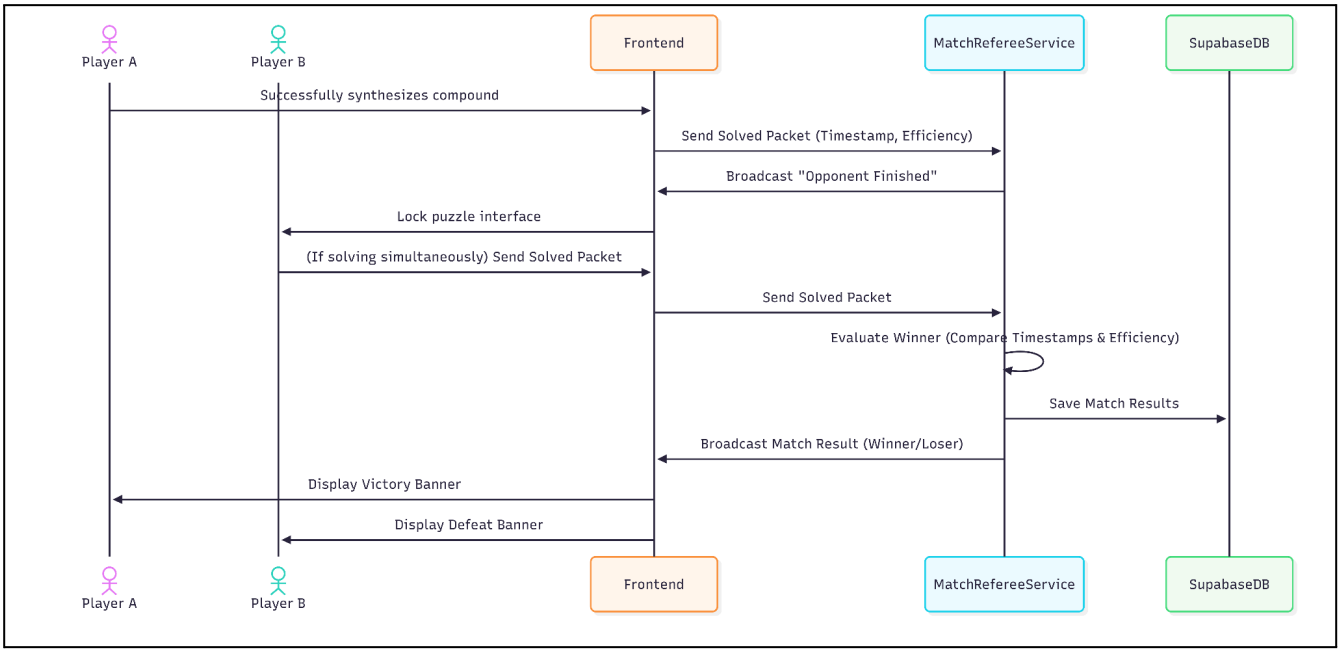



- Front-end component(s)
  - SpeedEvaluatorClient
    - Detects a valid compound synthesis and instantly transmits a 'solved' packet (including timestamp and efficiency) to the backend. Locks the interface upon receiving an opponent's victory signal.
    - React Hook / Context API
  - MatchResultBanner
    - Displays the outcome of the match (Win/Loss/Draw) based on the backend evaluation.
    - React UI Component
- Back-end component(s)
  - MatchRefereeService
    - Receives completion packets from both players. Compares timestamps and reaction path efficiency to determine the winner with 100% computational accuracy. Handles tie-breakers (failed attempts).
    - Spring Boot Service Class
- Object-Oriented Components

- Class Diagram




- Sequence Diagram



- Data Design
    - ERD or schema

| MATCH_COMPLETION | | | |
|---|---|---|---|
| string | completionId | PK | Unique UUID for completion record |
| string | matchId | FK | Links to match lobby |
| string | temporaryId | FK | Links to SESSION_NICKNAME |
| string | opponentId | | Competitor's nickname |
| string | matchResult | | WIN, LOSS, or DRAW |
| int | totalTimeElapsed | | Match duration in seconds |
| int | failedAttempts | | Count of incorrect actions |
| float | reactionPathEfficiency | | Calculated algorithmic score |
| timestamp | completedAt | | Precise end time |

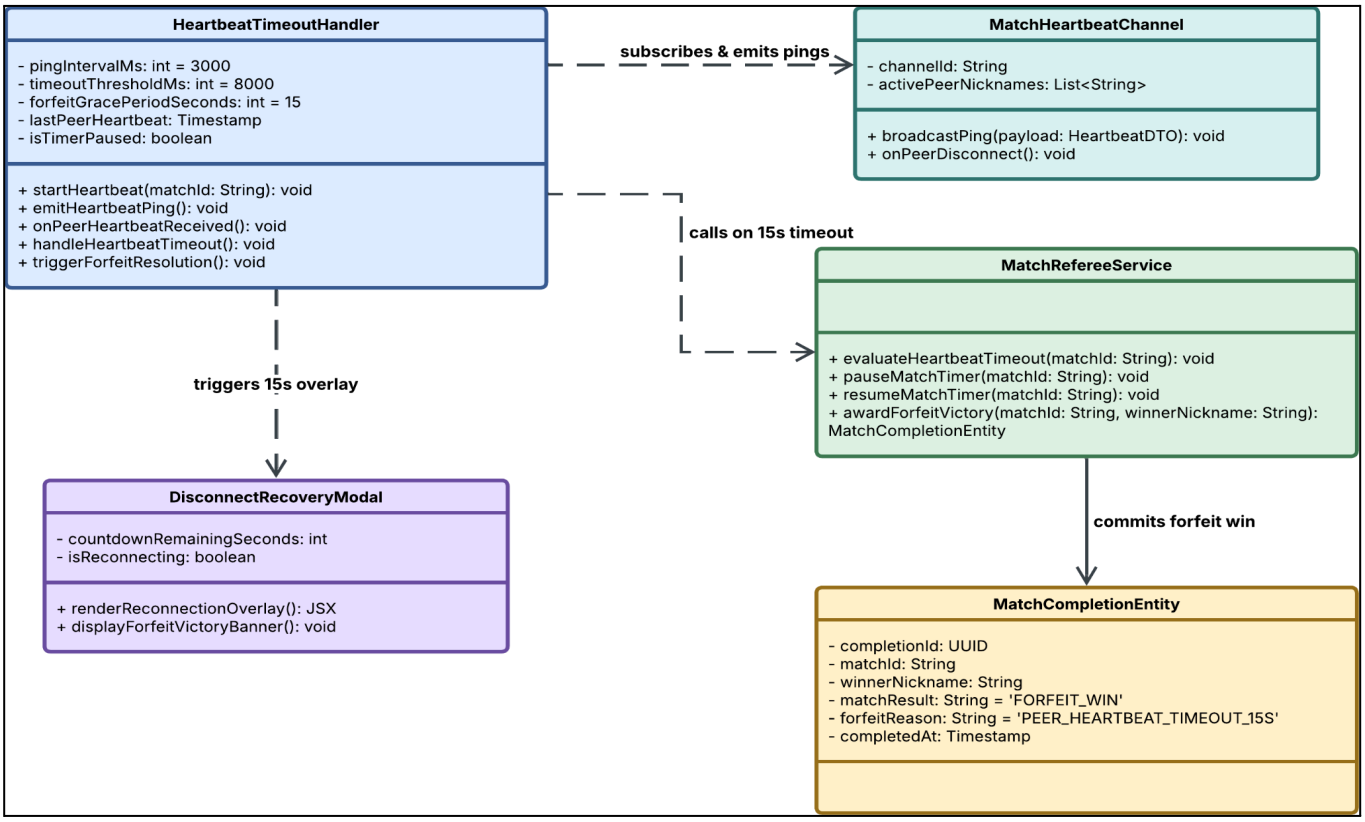
### *3.3.4 Realtime Heartbeat and Disconnect Engine*

- **User Interface Design:**

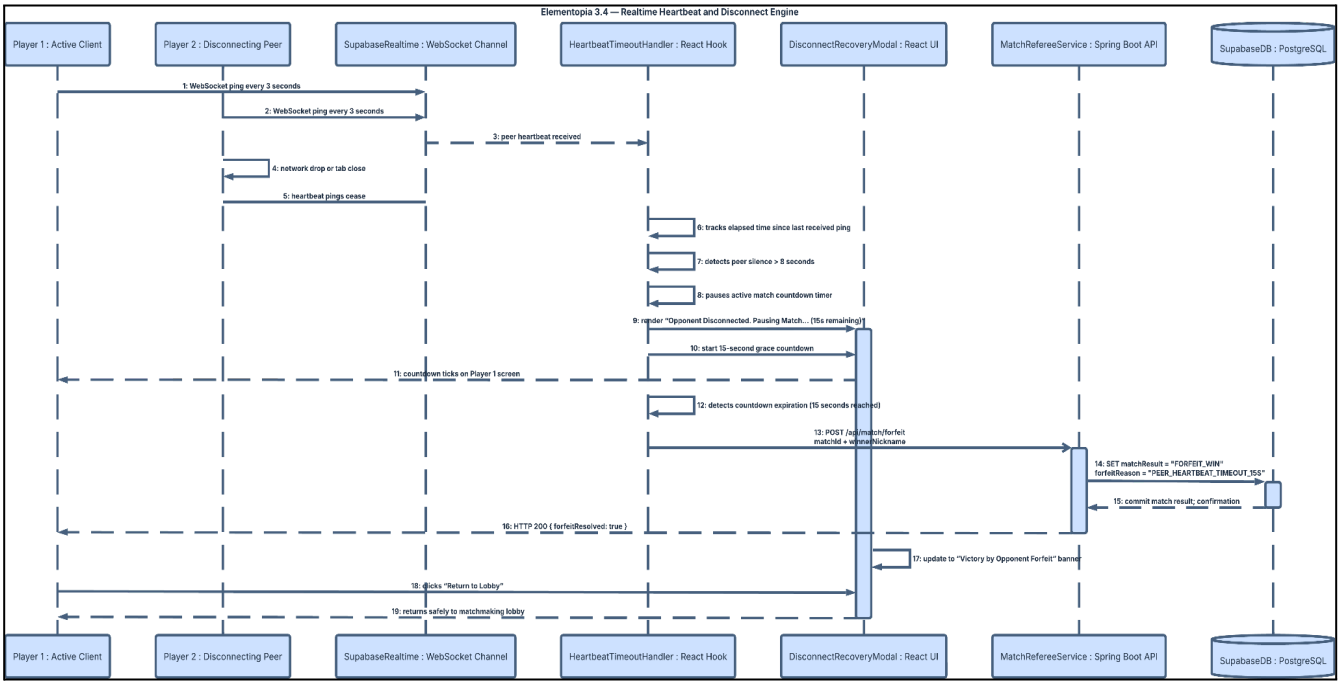



- When peer silence exceeds 8 seconds, renders a modal: "Opponent Disconnected. Pausing Match... (15s)" with a countdown progress bar. If unrecovered, renders a "Victory by Opponent Forfeit" banner.
- **Front-end component(s):**
  - Component Name: HeartbeatTimeoutHandler.js
    - Description and Purpose: Listens to Supabase Realtime presence channel. Emits WebSocket heartbeat every 3 seconds. Triggers pause state at 8s silence and invokes forfeit callback at 15s.
  - Component type or format: React Custom Hook / Realtime Listener.
  - Component Name: DisconnectRecoveryModal.jsx
    - Description and Purpose: Displays the reconnection countdown overlay and disables puzzle interactions while the match timer is paused.
  - Component type or format: Presentational React UI Component.
- **Back-end component(s):**
  - Component Name: MatchRefereeService.java
    - Description and Purpose: Validates forfeit events. If a player fails to reconnect within 15 seconds, marks match status as FORFEIT_WIN in Supabase without corrupting stats.
  - Component type or format: Spring Boot @Service class.
- Object-Oriented Component(s):

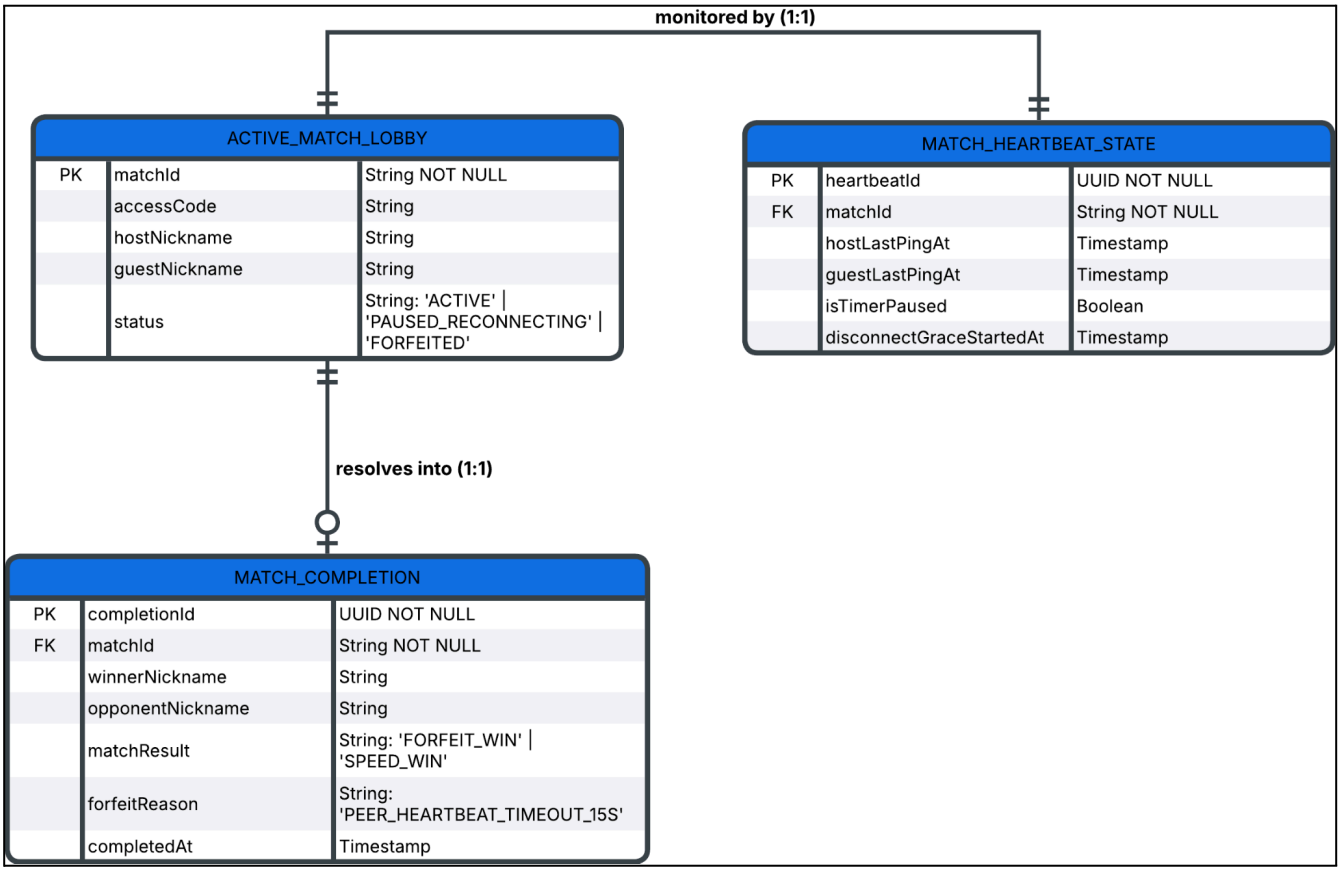
- Class Diagram




- Sequence Diagram



- Data Design
    - ERD or schema



# 4. Cross-Cutting Design: Mobile Touch & Offline Resilience

## *4.1. Mobile Responsive Viewport & 48px Touch Target Standard*

To eliminate the mobile viewport scaling and mis-click issues identified during validation testing, the UI implements strict CSS responsive architecture:

- **Minimum 48px Bounding Box:** All draggable element tiles in ActiveInventoryTray.jsx and GameBoard.jsx enforce a minimum bounding box of 48px x 48px with 8px touch margins, adhering to WCAG 2.1 touch accessibility guidelines.
- **Touch Action Constraint:** CSS touch-action: none is applied to active workbench tiles, preventing unintended viewport scrolling or pinching during drag-and-drop interactions on iOS Safari and Android Chrome.
- **Responsive Viewport Breakpoints:** Viewport widths between 360px and 768px utilize a 2-column or 3-column auto-wrapping CSS grid layout with overflow-x: hidden to eliminate horizontal scrolling.

## *4.2. Offline FIFO LocalStorage Sync Manager*

To resolve the telemetry isolation gap discovered during MVP testing, the client architecture integrates an automated background sync manager:

- **Queueing Mechanism:** Whenever an in-game pre-test or room clear event occurs, the payload is immediately committed to a browser localStorage FIFO queue (elementopia_sync_queue).
- **Automatic Cloud Reconciliation:** The client registers a window.addEventListener('online') listener. Upon connection restoration, the queue flushes pending payloads via asynchronous POST requests to the Spring Boot / Supabase API, providing seamless offline-first graceful degradation.